QUANTUM SECURITY INTELLIGENCE

Security & Post-Quantum Readiness Audit Report

krux @ 9b3a4314a205 · Krux (selfcustody) — public repository (passive review)

Organization	QSI
Report date	2026-11-04 18:52 UTC
Report version	1
Audit run	59875687-bb02-418b-9798-bc32732ba918 · Document audit · 2026-11-04 18:52 UTC
Analyzer engine	qsi-engine 1.3.0
Risk profile	Crypto exchange, custodian or wallet
Content SHA-256	`e013074c7b0eda8ff8256cc33f333800e595ae0c48705d84b95076950e74be8d`
Signature key (Ed25519, SHA-256 fingerprint; the signature is the separate file report.sig.json)	`5b3cf9c9bd0fa54447e2b5dfa7a6bab569bd0c666fa46949599e5c348d17083e`

Contents

1. Executive summary
2. Scope and boundaries
3. Methodology
4. Assets examined and not examined
5. Architecture overview
6. Overall security posture
7. Findings by severity
8. Detailed findings
9. Evidence
10. Post-quantum assessment
11. Limitations and assumptions
12. Remediation plan
13. Priorities for 30, 60 and 90 days
14. Verification after remediation
15. Tools used
16. Sources and normative documents
17. Appendices

This report presents automated and documented analysis results for the stated scope. It is not a legal opinion, a certification or an attestation of compliance with any standard.

Executive summary

11	0	2	6	0
Findings in total	Critical	High	Medium	Low

Classification	Count
Confirmed technical vulnerability	0
Known vulnerable component	0
Confirmed configuration error	0
Potential cryptographic risk	4
Long-term post-quantum risk	6
Improvement recommendation	1
Insufficient data	0

Top 5

ID	Severity	Detailed findings	Priority score
QSI-2026-11969	High	ECB block-cipher mode used in src/krux/kef.py confirmed by the analyst	45.6
QSI-2026-11978	High	Non-cryptographic generator next to key or seed generation in vendor/embit/src/embit/util/key.py preliminary	38.8
QSI-2026-11972	Medium	Quantum-vulnerable ECDSA: transaction and message signing (31 file(s)) confirmed by the analyst	41.8
QSI-2026-11974	Medium	Quantum-vulnerable Schnorr (BIP-340): transaction and message signing (7 file(s)) confirmed by the analyst	41.8
QSI-2026-11975	Medium	Quantum-vulnerable Pedersen commitment: threshold signing and MPC protocols (4 file(s)) confirmed by the analyst	41.8

Mosca's inequality (x + y > z): x = 10, y = 5, z = 8 → migration would finish too late under this scenario. z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

Analyst review: 7 confirmed, 3 not applicable, 0 dismissed as false positives.

Post-quantum maturity: Level 0 of 4 — no post-quantum cryptography. Next step: Inventory where classical public-key cryptography protects long-lived data and keys, and choose the first migration: usually a hybrid TLS key exchange and wrapped keys.

Scope and boundaries

Mode: Document audit

Repository: https://github.com/selfcustody/krux @ 9b3a4314a205767606b1ad9fbd1da06ad908f38e — evidence links open the lines at this commit.

Supplied inputs

246 files, 2857956 bytes in total, 0 redaction(s); the per-file SHA-256 list is in appendix C.

Methodology

The audit was performed by the QSI engine 1.3.0. Each analyzer emits facts (rule, parameters, evidence); finding texts are generated from the rule catalog. Methodology sources are listed under "Sources and normative documents".

Prioritization uses an explainable weighted model (severity, exploitability, exposure, confidentiality/integrity/availability impact weighted by the "Crypto exchange, custodian or wallet" profile, asset criticality, urgency, ease of remediation) with a confidence multiplier. CVSS v3.1 is applied only to technical vulnerabilities with a defined vector. Long-term quantum risks use Mosca's inequality; their severity is capped at Medium and they are never presented as exploitable vulnerabilities.

Finding classes: confirmed technical vulnerability; known vulnerable component (a dependency version named in an advisory; whether the affected code is reachable is assessed separately, and a component confirmed by review becomes a vulnerability); confirmed configuration error; potential cryptographic risk; long-term post-quantum risk; improvement recommendation; insufficient data. Heuristic findings are marked preliminary; analyst decisions are shown next to the findings.

Coverage of checks

Area	Status	Count	Examples
dependencies	checked	3	pyproject.toml: 2 dependencies parsed (0 pinned); vendor/embit/poetry.lock: 55 dependencies parsed (55 pinned); vendor/embit/pyproject.toml: 0 dependencies parsed (0 pinned)
source_code	checked	243	src/boot.py: 182 lines scanned with 16 rule patterns and 145 registry patterns; src/krux/auto_shutdown.py: 79 lines scanned with 16 rule patterns and 145 registry patterns; src/krux/baseconv.py: 192 lines scanned with 16 rule patterns and 145 registry patterns
tls_key_exchange_groups	Insufficient data	56	vendor/embit/poetry.lock line 9: not in the algorithm registry, not assessed: ["main"]; vendor/embit/poetry.lock line 25: not in the algorithm registry, not assessed: ["main"]; vendor/embit/poetry.lock line 46: not in the algorithm registry, not assessed: ["main"]
tls_configuration	checked	2	vendor/embit/poetry.lock: openssl TLS directives parsed; vendor/embit/src/embit/slip39.py: openssl TLS directives parsed
entropy	checked	1	random-number generation: 243 code files read with 968 macro definitions from the tree; 0 ineffective guard(s), 0 weak random-bytes provider(s), 1 weak key-generation site(s)
osv_advisories	checked	1	55 pinned packages queried at OSV.dev; 8 affected (8 of them only in development or build dependencies)
osv_advisories	Insufficient data	1	PyPI/virtualenv@21.2.0: 23 advisories at OSV.dev, the first 20 are listed as findings; see OSV.dev for the rest

Long lists are grouped here; the full per-file list is in the JSON report, signed with the same key.

Assets examined and not examined

Supplied inputs: 248 (input files: C)

Not checked

- tls_key_exchange_groups: 56 (examples: vendor/embit/poetry.lock line 9: not in the algorithm registry, not assessed: ["main"]; vendor/embit/poetry.lock line 25: not in the algorithm registry, not assessed: ["main"]; vendor/embit/poetry.lock line 46: not in the algorithm registry, not assessed: ["main"])
- osv_advisories: 1 (examples: PyPI/virtualenv@21.2.0: 23 advisories at OSV.dev, the first 20 are listed as findings; see OSV.dev for the rest)

Architecture overview

Static review of the source tree krux at 9b3a4314a205767606b1ad9fbd1da06ad908f38e. 246 files analysed; of 437 files, 71 are test, fuzzing, benchmark or example code and were left out, 120 are not read by any analyzer, 0 exceed the per-file limit and 0 did not fit the total limit. The review was restricted to 6 path(s) of the repository: .coveragerc, .gitignore, firmware/Kboot, pyproject.toml, src, vendor; 8852 files outside them were not read. Files no analyzer reads include: .coveragerc, .gitignore, firmware/Kboot/.git, firmware/Kboot/.github/FUNDING.yml, firmware/Kboot/.gitignore. 159 files come from directories a .gitmodules pins and are read as shipped source rather than skipped as a dependency cache: vendor/embit (142), firmware/Kboot (17). No requests were sent to the owner's systems.

Overall security posture

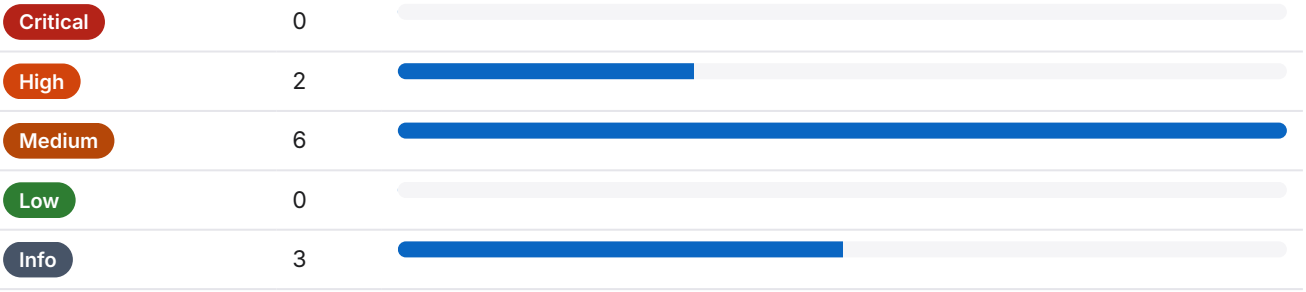
81
Indicative posture index (0–100)

63.4%
Quantum-vulnerable share of inventory

93
Cryptographic inventory

Derived from findings in scope (confirmed and potential issues weighted by severity). Indicative only; not comparable across differently scoped audits.

Findings by severity



ID	Detailed findings	Classification	Severity	Confidence
QSI-2026-11969	ECB block-cipher mode used in src/krux/kef.py	Potential cryptographic risk	High	Medium
QSI-2026-11978	Non-cryptographic generator next to key or seed generation in vendor/embit/src/embit/util/key.py	Potential cryptographic risk	High	Medium, preliminary
QSI-2026-11972	Quantum-vulnerable ECDSA: transaction and message signing (31 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-11974	Quantum-vulnerable Schnorr (BIP-340): transaction and message signing (7 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-11975	Quantum-vulnerable Pedersen commitment: threshold signing and MPC protocols (4 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-11970	Quantum-vulnerable ECDSA: role not determined (5 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-11973	Quantum-vulnerable ECDH: role not determined (4 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-11977	Quantum-vulnerable Schnorr (BIP-340): role not determined (1 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-11971	ECB block-cipher mode used in src/krux/krux_settings.py	Potential cryptographic risk	Info	Medium
QSI-2026-11976	Low PBKDF2 iteration count (2500) in vendor/embit/src/embit/slip39.py	Potential cryptographic risk	Info	Medium
QSI-2026-11979	40 advisories in 8 development or build-only PyPI package(s)	Improvement recommendation	Info	Medium

Detailed findings

QSI-2026-11969 · CODE-ECB-MODE

High

ECB block-cipher mode used in src/krux/kef.py

Potential cryptographic risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-29): confirmed by the analyst.** Real and deliberate: src/krux/kef.py defines the Krux Encrypted Format, and of its eleven versions four are ECB — version 0 "AES-ECB v1" (released 2023.08 for BIP-39 entropy), 5 "AES-ECB", 6 "AES-ECB +p" and 7 "AES-ECB +c". So a user can store an encrypted mnemonic under ECB, and old backups already are. Reported at medium rather than high, because the usual reasons for calling ECB high do not all hold here and saying so is the point of a review. Every version carries an `auth` field sizing an authentication tag, so integrity is not left to the mode; GCM versions 20 and 21 exist in the same table; and the maintainers document the actual hazard themselves, version 6 reading "safe padding: for mid-sized plaintext w/o duplicate blocks" and 7 "may compact otherwise duplicate blocks". For the intended payload — one or two blocks of high-entropy mnemonic — ECB's block-repetition leak has little to bite on. What remains true, and is the actionable half: versions 6 and 7 exist precisely for larger plaintext, which is where duplicate blocks become possible, and ECB there leaks structure that CTR or GCM would not. The recommendation is not "remove ECB" (that would orphan 2023 backups) but to keep the ECB versions for decryption of old backups and stop offering them for new ones.

8 usage(s) of ECB mode (first at line 30).

Affected asset	file:src/krux/kef.py
Data source	src/krux/kef.py
Detection method	Regular-expression matching of language-specific API patterns.
Analyzer	source_code 1.3.0 · 2026-11-04 18:52 UTC
Priority score	45.6 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable

POTENTIAL IMPACT

ECB reveals patterns in encrypted data and provides no integrity.

REPRODUCTION STEPS

```
Open line 30 in the supplied file src/krux/kef.py and inspect the quoted lines (see evidence).
```

EVIDENCE

file_excerpt · src/krux/kef.py · line 30 · SHA-256 983f2cc48989ecc4… · File deb09c47bf209986…

```
28 |  #  to disable a VERSION: set its value to None
29 |  AES = ucryptolib.aes
30 |  MODE_ECB = ucryptolib.MODE_ECB
31 |  MODE_CBC = ucryptolib.MODE_CBC
32 |  MODE_CTR = ucryptolib.MODE_CTR
```

file_excerpt · src/krux/kef.py · line 38 · SHA-256 e840c4c38d9dd065… · File deb09c47bf209986…

```
   36 |     0: {
   37 |         "name": "AES-ECB v1",
   38 |         "mode": MODE_ECB,
   39 |         "auth": -16,
   40 |     },
```

file_excerpt · src/krux/kef.py · line 50 · SHA-256 4102c93b12405028… · File deb09c47bf209986…

```
   48 |         # smallest ECB ciphertext, w/ unsafe padding: for high entropy mnemonics,
passphrases, etc
   49 |         "name": "AES-ECB",
   50 |         "mode": MODE_ECB,
   51 |         "auth": 3,
   52 |     },
```

file_excerpt · src/krux/kef.py · line 56 · SHA-256 2b53d9a1ca1d33e5… · File deb09c47bf209986…

```
   54 |         # safe padding: for mid-sized plaintext w/o duplicate blocks
   55 |         "name": "AES-ECB +p",
   56 |         "mode": MODE_ECB,
   57 |         "pkcs_pad": True,
   58 |         "auth": -4,
```

file_excerpt · src/krux/kef.py · line 63 · SHA-256 dbd2e22444870467… · File deb09c47bf209986…

```
   61 |         # compressed, w/ safe padding: for larger plaintext; may compact otherwise duplicate
blocks
   62 |         "name": "AES-ECB +c",
   63 |         "mode": MODE_ECB,
   64 |         "pkcs_pad": True,
   65 |         "auth": -4,
```

LIMITATIONS OF THE CHECK

Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Use an AEAD mode (GCM) with unique nonces.

RETEST PROCEDURE

Re-scan the source and review the call sites listed in the evidence.

NIST SP 800-131A Rev. 2 — Transitioning the Use of Cryptographic Algorithms and Key Lengths

QSI-2026-11978 · RNG-WEAK-KEYGEN

High

Non-cryptographic generator next to key or seed generation in vendor/embit/src/embit/util/key.py

Potential cryptographic risk | Confidence: Medium | preliminary — requires further verification

'random.randrange(' at line 374, within a few lines of 'generate_privkey' (1 occurrence(s)).

Affected asset	vendor/embit/src/embit/util/key.py:keygen
Data source	vendor/embit/src/embit/util/key.py
Detection method	Regular-expression matching of language-specific API patterns.
Analyzer	entropy 1.3.0 · 2026-11-04 18:52 UTC

Priority score	38.8 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable

POTENTIAL IMPACT

Mnemonics, seeds or private keys drawn from such a generator can be enumerated from its small internal state: Milk Sad (CVE-2023-39910, mt19937 behind 'bx seed') and Trust Wallet (CVE-2023-31290) lost funds this way.

REPRODUCTION STEPS

```
Open line 374 in the supplied file vendor/embit/src/embit/util/key.py and inspect the quoted lines
(see evidence).
```

EVIDENCE

file_excerpt · vendor/embit/src/embit/util/key.py · line 374 · SHA-256 6a16bf61b745add2... · File 0850b1637dafc13c...

```
372 | def generate_privkey():
373 |     """Generate a valid random 32-byte private key."""
374 |     return random.randrange(1, SECP256K1_ORDER).to_bytes(32, "big")
375 |
376 |
```

LIMITATIONS OF THE CHECK

Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Use the platform CSPRNG (secrets, os.urandom, crypto.getRandomValues, crypto/rand, SecureRandom, OsRng) or a hardware TRNG for every key, seed and mnemonic.

RETEST PROCEDURE

Re-scan the source and review the call sites listed in the evidence.

CWE-338 — Use of Cryptographically Weak Pseudo-Random Number Generator (PRNG); CVE-2023-39910 — Libbitcoin Explorer 'bx seed' entropy (Milk Sad): mt19937 limits it to 32 bits; CVE-2023-31290 — Trust Wallet Core: 32 bits of mnemonic entropy (mt19937 seeded with one 32-bit value)

QSI-2026-11972 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable ECDSA: transaction and message signing (31 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-29): confirmed by the analyst.** Correct, and this is the finding the audit exists to produce. 43 of the 87 inventory rows are ECDSA (53 of 98 as first published on 2026-10-29, before eleven benchmark files of the vendored secp256k1-zkp were set aside) and they span the whole chain: src/krux/key.py:255 calls secp256k1.ecdsa_sign_recoverable through embit, which dispatches in vendor/embit/src/embit/util/secp256k1.py either to ctypes_secp256k1.py (the C library) or to py_secp256k1.py (a pure-Python fallback), and the C library itself is vendored again at vendor/embit/secp256k1/secp256k1-zkp/. Every spend this device authorises is a discrete-log signature over secp256k1, so Shor applies to the signing path in full. Forward-only rather than retroactive: a signature already published cannot be made more secret, but a public key that has been seen

> on chain can be attacked the day a CRQC exists, which is why address reuse is the behaviour worth changing before the scheme does.

277 reference(s) to ECDSA across 31 file(s), e.g. vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_parsing.c:11. Role, judged from paths, function names and nearby code: transaction and message signing. A CRQC derives the private key from a public key that is already known (on chain after the first spend, in reused addresses): funds and approvals controlled by these keys need a migration path.

Affected asset	source-code:ECDSA:transaction_signing
Data source	vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_parsing.c
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-04 18:52 UTC
Priority score	41.8 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_parsing.c:11 and inspect the quoted lines;
the evidence lists further files.
```

EVIDENCE

file_excerpt · vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_parsing.c · line 11 · SHA-256 9197354c850cb53b… · File 14225148538be558…

```
   10 |
   11 | int ecdsa_signature_parse_der_lax(const secp256k1_context* ctx, secp256k1_ecdsa_signature*
sig, const unsigned char *input, size_t inputlen) {
   12 |     size_t rpos, rlen, spos, slen;
```

file_excerpt · vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_parsing.c · line 19 · SHA-256 234aeaaf6b28e1d2… · File 14225148538be558…

```
   18 |     /* Hack to initialize sig with a correctly-parsed but invalid signature. */
   19 |     secp256k1_ecdsa_signature_parse_compact(ctx, sig, tmpsig);
   20 |
```

file_excerpt · vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_parsing.c · line 140 · SHA-256 75ad5d0cba25ca64… · File 14225148538be558…

```
  139 |     if (!overflow) {
  140 |         overflow = !secp256k1_ecdsa_signature_parse_compact(ctx, sig, tmpsig);
  141 |     }
```

file_excerpt · vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_parsing.h · line 57 · SHA-256 25af7e242d92af0d… · File 79c479a9e6f51a5a…

```
56 |    files (such as tests.c) without the need to set an explicit -I flag,
57 |    which would be necessary to locate secp256k1.h. */
58 | #ifndef SECP256K1_H
```

file_excerpt · vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_parsing.h · line 59 · SHA-256 4c35a81e99a7eaf5… · File 79c479a9e6f51a5a…

```
58 | #ifndef SECP256K1_H
59 | #include <secp256k1.h>
60 | #endif
```

file_excerpt · vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_parsing.h · line 88 · SHA-256 85efcade66268dd4… · File 79c479a9e6f51a5a…

```
87 |     const secp256k1_context* ctx,
88 |     secp256k1_ecdsa_signature* sig,
89 |     const unsigned char *input,
```

file_excerpt · vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_privatekey_parsing.h · line 34 · SHA-256 163cac24f6b60e79… · File 3dcaf6d98270a20e…

```
33 |    files (such as tests.c) without the need to set an explicit -I flag,
34 |    which would be necessary to locate secp256k1.h. */
35 | #ifndef SECP256K1_H
```

file_excerpt · vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_privatekey_parsing.h · line 36 · SHA-256 2741d1869779ddac… · File 3dcaf6d98270a20e…

```
35 | #ifndef SECP256K1_H
36 | #include <secp256k1.h>
37 | #endif
```

file_excerpt · vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1.h · line 85 · SHA-256 976a23a4a733f368… · File 524e48b39a62c9f5…

```
84 |     unsigned char data[64];
85 | } secp256k1_ecdsa_signature;
86 |
```

file_excerpt · vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1.h · line 417 · SHA-256 15748a6fd21f774c… · File 524e48b39a62c9f5…

```
416 |  */
417 | SECP256K1_API int secp256k1_ecdsa_signature_parse_compact(
418 |     const secp256k1_context* ctx,
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains adopt them.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11974 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable Schnorr (BIP-340): transaction and message signing (7 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-29): confirmed by the analyst.** Correct. Six rows (seven as first published, one of them in a benchmark file): vendor/embit/src/embit/util/secp256k1.py:33 and ctypes_secp256k1.py:287, and the vendored library's own schnorrsig and musig modules. Taproot spends therefore sit in the same class as ECDSA ones: BIP-340 changes the signature's shape, not the hardness assumption it rests on.

21 reference(s) to Schnorr (BIP-340) across 7 file(s), e.g. vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_schnorrsig.h:82. Role, judged from paths, function names and nearby code: transaction and message signing. A CRQC derives the private key from a public key that is already known (on chain after the first spend, in reused addresses): funds and approvals controlled by these keys need a migration path.

Affected asset	source-code:Schnorr (BIP-340):transaction_signing
Data source	vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_schnorrsig.h
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-04 18:52 UTC
Priority score	41.8 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_schnorrsig.h:82 and inspect the quoted lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_schnorrsig.h · line 82 · SHA-256 44a2365db8154794… · File 4c06e835211e9426…

```
 81 |  */
 82 | SECP256K1_API int secp256k1_schnorrsig_sign(
 83 |     const secp256k1_context* ctx,
```

file_excerpt · vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_schnorrsig.h · line 100 · SHA-256 72dc7958413515e7… · File 4c06e835211e9426…

```
 99 |  */
100 | SECP256K1_API SECP256K1_WARN_UNUSED_RESULT int secp256k1_schnorrsig_verify(
101 |     const secp256k1_context* ctx,
```

file_excerpt · vendor/embit/secp256k1/secp256k1-zkp/src/modules/musig/example.c · line 160 · SHA-256 e6ab7181c8dbb650… · File 7568f0ec3778da01…

```
159 |     printf("Verifying signature.....");
160 |     if (!secp256k1_schnorrsig_verify(ctx, sig, msg, &combined_pk)) {
161 |         printf("FAILED\n");
```

file_excerpt · vendor/embit/secp256k1/secp256k1-zkp/src/modules/musig/main_impl.h · line 460 · SHA-256 8347c1cd3e521a8b… · File f5b1e1cb467ee05a…

```
459 |
460 |     secp256k1_schnorrsig_sha256_tagged(&sha);
461 |     secp256k1_xonly_pubkey_load(ctx, &rp, &session->combined_nonce);
```

file_excerpt · vendor/embit/secp256k1/secp256k1-zkp/src/modules/schnorrsig/main_impl.h · line 98 · SHA-256 e73ca8d800b70a37… · File eb51e55a0617e614…

```
 97 |  * SHA256 to SHA256("BIP0340/challenge")||SHA256("BIP0340/challenge"). */
 98 | static void secp256k1_schnorrsig_sha256_tagged(secp256k1_sha256 *sha) {
 99 |     secp256k1_sha256_initialize(sha);
```

file_excerpt · vendor/embit/secp256k1/secp256k1-zkp/src/modules/schnorrsig/main_impl.h · line 111 · SHA-256 0dd4fa6557933bdf… · File eb51e55a0617e614…

```
110 |
111 | static void secp256k1_schnorrsig_challenge(secp256k1_scalar* e, const unsigned char *r32,
const unsigned char *msg32, const unsigned char *pubkey32)
112 | {
```

file_excerpt · vendor/embit/secp256k1/secp256k1-zkp/src/modules/schnorrsig/main_impl.h · line 117 · SHA-256 d433a64049d5b15a… · File eb51e55a0617e614…

```
116 |     /* tagged hash(r.x, pk.x, msg32) */
117 |     secp256k1_schnorrsig_sha256_tagged(&sha);
118 |     secp256k1_sha256_write(&sha, r32, 32);
```

file_excerpt · vendor/embit/src/embit/util/ctypes_secp256k1.py · line 287 · SHA-256 57e08893e8b0d02f… · File 76695a82303a7699…

```
286 |
287 |         secp256k1.secp256k1_schnorrsig_verify.argtypes = [
288 |             c_void_p,  # ctx
```

file_excerpt · vendor/embit/src/embit/util/ctypes_secp256k1.py · line 293 · SHA-256 081fdf562b7a5fc8… · File 76695a82303a7699…

```
292 |         ]
293 |         secp256k1.secp256k1_schnorrsig_verify.restype = c_int
294 |
```

file_excerpt · vendor/embit/src/embit/util/ctypes_secp256k1.py · line 295 · SHA-256 ca6ef88f4a54753a… · File 76695a82303a7699…

```
294 |
295 |         secp256k1.secp256k1_schnorrsig_sign.argtypes = [
296 |             c_void_p,  # ctx
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains adopt them.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11975 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable Pedersen commitment: threshold signing and MPC protocols (4 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-29): confirmed by the analyst.** Correct but narrow, and the narrowness is worth recording so nobody reads this as four more rows of spending exposure. All four are Liquid, not Bitcoin: vendor/embit/src/embit/liquid/pset.py:101 and liquid/transaction.py:90 call secp256k1.pedersen_commit for confidential-transaction value commitments, via util/secp256k1.py:51 and ctypes_secp256k1.py:949. The property at risk is the commitment's *binding*, which rests on the discrete log, so a CRQC could forge a value commitment; the *hiding* property is information-theoretic and no quantum computer touches it. A post-quantum signature standard does not fix this — it needs a different commitment scheme — and it only matters to users who spend Liquid assets.

15 reference(s) to Pedersen commitment across 4 file(s), e.g. vendor/embit/src/embit/liquid/pset.py:101. Role, judged from paths, function names and nearby code: threshold signing and MPC protocols. Paillier, Pedersen commitments and the discrete-logarithm proofs inside threshold ECDSA and EdDSA protocols all fall to Shor's algorithm; a post-quantum threshold scheme is a protocol change, not a parameter change.

Affected asset	source-code:Pedersen commitment:threshold_mpc
Data source	vendor/embit/src/embit/liquid/pset.py
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-04 18:52 UTC
Priority score	41.8 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open vendor/embit/src/embit/liquid/pset.py:101 and inspect the quoted lines; the evidence lists
further files.
```

EVIDENCE

file_excerpt · vendor/embit/src/embit/liquid/pset.py · line 101 · SHA-256 b66b1e583d90306b... · File b9bd8b713c639528...

```
100 |             raise PSBTError("Invalid asset commitment")
101 |         cmt = secp256k1.pedersen_commit(vbf, value, gen)
102 |         if cmt != secp256k1.pedersen_commitment_parse(self.utxo.value):
```

file_excerpt · vendor/embit/src/embit/liquid/pset.py · line 102 · SHA-256 06f441e13a3c4ee8... · File b9bd8b713c639528...

```
101 |         cmt = secp256k1.pedersen_commit(vbf, value, gen)
102 |         if cmt != secp256k1.pedersen_commitment_parse(self.utxo.value):
103 |             raise PSBTError("Invalid value commitment")
```

file_excerpt · vendor/embit/src/embit/liquid/pset.py · line 300 · SHA-256 e13d6f4ff6fe3bd7... · File b9bd8b713c639528...

```
299 |             if self.value_blinding_factor:
300 |                 value_commitment = secp256k1.pedersen_commit(
301 |                     self.value_blinding_factor, self.value, gen
```

file_excerpt · vendor/embit/src/embit/liquid/transaction.py · line 90 · SHA-256 ffe62aaa23ace0fd... · File 601e672e0ae53c08...

```
 89 |
 90 |     commit = secp256k1.pedersen_commitment_parse(value_commitment)
 91 |     gen = secp256k1.generator_parse(asset_commitment)
```

file_excerpt · vendor/embit/src/embit/util/ctypes_secp256k1.py · line 949 · SHA-256 b7bf98d515fa733b... · File 76695a82303a7699...

```
948 | @locked
949 | def pedersen_commitment_parse(inp, context=_secp.ctx):
950 |     if len(inp) != 33:
```

file_excerpt · vendor/embit/src/embit/util/ctypes_secp256k1.py · line 960 · SHA-256 ae7600847f2badfd... · File 76695a82303a7699...

```
959 | @locked
960 | def pedersen_commitment_serialize(commit, context=_secp.ctx):
961 |     if len(commit) != 64:
```

file_excerpt · vendor/embit/src/embit/util/ctypes_secp256k1.py · line 971 · SHA-256 d061be5d1ea86ffb... · File 76695a82303a7699...

```
970 | @locked
971 | def pedersen_commit(vbf, value, gen, context=_secp.ctx):
972 |     if len(gen) != 64:
```

file_excerpt · vendor/embit/src/embit/util/secp256k1.py · line 51 · SHA-256 4485c26fa3e644b5... · File 3f0dfccabac80a6c...

```
 50 |             "generator_serialize": "secp256k1_generator_serialize",
 51 |             "pedersen_commitment_parse": "secp256k1_pedersen_commitment_parse",
 52 |             "pedersen_commitment_serialize": "secp256k1_pedersen_commitment_serialize",
```

file_excerpt · vendor/embit/src/embit/util/secp256k1.py · line 52 · SHA-256 9ee2ecc76d93cb9f... · File 3f0dfccabac80a6c...

```
51 |             "pedersen_commitment_parse": "secp256k1_pedersen_commitment_parse",
52 |             "pedersen_commitment_serialize": "secp256k1_pedersen_commitment_serialize",
53 |             "pedersen_commit": "secp256k1_pedersen_commit",
```

file_excerpt · vendor/embit/src/embit/util/secp256k1.py · line 53 · SHA-256 b419e6b04ad1f224… · File 3f0dfccabac80a6c…

```
52 |             "pedersen_commitment_serialize": "secp256k1_pedersen_commitment_serialize",
53 |             "pedersen_commit": "secp256k1_pedersen_commit",
54 |             "pedersen_blind_generator_blind_sum": (
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Track post-quantum threshold signatures (NIST IR 8214C) and keep the protocol layer ready to swap the signature scheme; record which keys and shares must stay valid for how long.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11970 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable ECDSA: role not determined (5 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-29): confirmed by the analyst.** Correct, and this is the finding the audit exists to produce. 43 of the 87 inventory rows are ECDSA (53 of 98 as first published on 2026-10-29, before eleven benchmark files of the vendored secp256k1-zkp were set aside) and they span the whole chain: src/krux/key.py:255 calls secp256k1.ecdsa_sign_recoverable through embit, which dispatches in vendor/embit/src/embit/util/secp256k1.py either to ctypes_secp256k1.py (the C library) or to py_secp256k1.py (a pure-Python fallback), and the C library itself is vendored again at vendor/embit/secp256k1/secp256k1-zkp/. Every spend this device authorises is a discrete-log signature over secp256k1, so Shor applies to the signing path in full. Forward-only rather than retroactive: a signature already published cannot be made more secret, but a public key that has been seen on chain can be attacked the day a CRQC exists, which is why address reuse is the behaviour worth changing before the scheme does.

34 reference(s) to ECDSA across 5 file(s), e.g. src/krux/key.py:252. Role, judged from paths, function names and nearby code: role not determined. The code does not show what the keys protect; review decides the priority.

Affected asset	source-code:ECDSA:general
Data source	src/krux/key.py

Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-04 18:52 UTC
Priority score	40.2 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open src/krux/key.py:252 and inspect the quoted lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · src/krux/key.py · line 252 · SHA-256 3571a45d6fb42550… · File f9176b1f127dae22…

```
251 |         from embit import ec
252 |         from embit.util import secp256k1
253 |
```

file_excerpt · src/krux/key.py · line 255 · SHA-256 4b287db1f3538b56… · File f9176b1f127dae22…

```
254 |         prv = self.root.derive(derivation).key
255 |         sig = secp256k1.ecdsa_sign_recoverable(
256 |             message_hash, prv._secret  # pylint: disable=W0212
```

file_excerpt · src/krux/key.py · line 261 · SHA-256 70e842829bb3f220… · File f9176b1f127dae22…

```
260 |         ec_signature = ec.Signature(sig[:64])
261 |         ser = flag + secp256k1.ecdsa_signature_serialize_compact(
262 |             ec_signature._sig  # pylint: disable=W0212
```

file_excerpt · vendor/embit/src/embit/ec.py · line 3 · SHA-256 2da5ac4f0aac6b95… · File 1c2b11a8e397a903…

```
  2 | from . import hashes
  3 | from .misc import secp256k1
  4 | from .networks import NETWORKS
```

file_excerpt · vendor/embit/src/embit/ec.py · line 18 · SHA-256 22af8149d1de86bd… · File 1c2b11a8e397a903…

```
 17 |     def write_to(self, stream) -> int:
 18 |         return stream.write(secp256k1.ecdsa_signature_serialize_der(self._sig))
 19 |
```

file_excerpt · vendor/embit/src/embit/ec.py · line 24 · SHA-256 91eebf765cedecb0… · File 1c2b11a8e397a903…

```
 23 |         der += stream.read(der[1])
 24 |         return cls(secp256k1.ecdsa_signature_parse_der(der))
 25 |
```

file_excerpt · vendor/embit/src/embit/misc.py · line 7 · SHA-256 05b74d409d615863… · File fc544d7dcb476386…

```
  6 |     from micropython import const
  7 |     import secp256k1
  8 | else:
```

file_excerpt · vendor/embit/src/embit/misc.py · line 9 · SHA-256 ff45e726cacf161b... · File fc544d7dcb476386...

```
 8 | else:
 9 |     from .util import secp256k1
10 |
```

file_excerpt · vendor/embit/src/embit/util/__init__.py · line 1 · SHA-256 00860f29eecbf17f... · File 23cccc8f03e64510...

```
1 | from . import secp256k1
2 |
```

file_excerpt · vendor/embit/src/embit/util/key.py · line 3 · SHA-256 fb27cc95d202f1f9... · File 0850b1637dafc13c...

```
2 | Copy-paste from key.py in bitcoin test_framework.
3 | This is a fallback option if the library can't do ctypes bindings to secp256k1 library.
4 | """
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Classify each usage, record data lifetimes, and plan migration to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it; introduce an abstraction layer for crypto-agility.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11973 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable ECDH: role not determined (4 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-29): confirmed by the analyst.** Kept rather than dismissed. The rule could not classify the role for five files, and that is honest: the anchor is src/krux/key.py:252-255, where the import of embit.util.secp256k1 and the call to ecdsa_sign_recoverable sit together, so the role is in fact transaction signing and the surrounding rows say so. Left as "role not determined" in the report because the classifier's own confidence should not be overwritten by the analyst's reading of the same lines; the decision records the reading instead.

9 reference(s) to ECDH across 4 file(s), e.g. vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_ecdh.h:49. Role, judged from paths, function names and nearby code: role not determined. The code does not show what the keys protect; review decides the priority.

Affected asset	source-code:ECDH:general

Data source	vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_ecdh.h
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-04 18:52 UTC
Priority score	40.2 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_ecdh.h:49 and inspect the quoted lines;
the evidence lists further files.
```

EVIDENCE

file_excerpt · vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_ecdh.h · line 49 · SHA-256 72841f4795f99327… · File 7950343b5eea7112…

```
   48 |  */
   49 | SECP256K1_API SECP256K1_WARN_UNUSED_RESULT int secp256k1_ecdh(
   50 |   const secp256k1_context* ctx,
```

file_excerpt · vendor/embit/secp256k1/secp256k1-zkp/src/modules/ecdh/main_impl.h · line 10 · SHA-256 65d8983e01c7b44d… · File b88931b1b2f71adb…

```
    9 |
   10 | #include "../../../include/secp256k1_ecdh.h"
   11 | #include "../../ecmult_const_impl.h"
```

file_excerpt · vendor/embit/secp256k1/secp256k1-zkp/src/modules/ecdh/main_impl.h · line 29 · SHA-256 99bef9ddb51d26be… · File b88931b1b2f71adb…

```
   28 |
   29 | int secp256k1_ecdh(const secp256k1_context* ctx, unsigned char *output, const
secp256k1_pubkey *point, const unsigned char *scalar, secp256k1_ecdh_hash_function hashfp, void
*data) {
   30 |     int ret = 0;
```

file_excerpt · vendor/embit/src/embit/util/ctypes_secp256k1.py · line 131 · SHA-256 25ed92dcf661ed23… · File 76695a82303a7699…

```
  130 |     preferred_symbols = (
  131 |         "secp256k1_ecdh",
  132 |         "secp256k1_xonly_pubkey_from_pubkey",
```

file_excerpt · vendor/embit/src/embit/util/ctypes_secp256k1.py · line 265 · SHA-256 de340041c8677f93… · File 76695a82303a7699…

```
  264 |     try:
  265 |         secp256k1.secp256k1_ecdh.argtypes = [
  266 |             c_void_p,  # ctx
```

file_excerpt · vendor/embit/src/embit/util/ctypes_secp256k1.py · line 273 · SHA-256 9d8302f1aac7f738… · File 76695a82303a7699…

```
272 |         ]
273 |         secp256k1.secp256k1_ecdh.restype = c_int
274 |     except Exception:  # noqa: S110
```

file_excerpt · vendor/embit/src/embit/util/secp256k1.py · line 31 · SHA-256 4be965210b8e21d1... · File 3f0dfccabac80a6c...

```
30 |         _OPTIONAL_SYMBOLS = {
31 |             "ecdh": "secp256k1_ecdh",
32 |             "xonly_pubkey_from_pubkey": "secp256k1_xonly_pubkey_from_pubkey",
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Classify each usage, record data lifetimes, and plan migration to Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH; introduce an abstraction layer for crypto-agility.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11977 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable Schnorr (BIP-340): role not determined (1 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-29): confirmed by the analyst.** Correct. Six rows (seven as first published, one of them in a benchmark file): vendor/embit/src/embit/util/secp256k1.py:33 and ctypes_secp256k1.py:287, and the vendored library's own schnorrsig and musig modules. Taproot spends therefore sit in the same class as ECDSA ones: BIP-340 changes the signature's shape, not the hardness assumption it rests on.

2 reference(s) to Schnorr (BIP-340) across 1 file(s), e.g. vendor/embit/src/embit/util/key.py:535. Role, judged from paths, function names and nearby code: role not determined. The code does not show what the keys protect; review decides the priority.

Affected asset	source-code:Schnorr (BIP-340):general
Data source	vendor/embit/src/embit/util/key.py
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-04 18:52 UTC
Priority score	40.2 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open vendor/embit/src/embit/util/key.py:535 and inspect the quoted lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · vendor/embit/src/embit/util/key.py · line 535 · SHA-256 0597a48161191ebf... · File 0850b1637dafc13c...

```
534 |
535 | def verify_schnorr(key, sig, msg):
536 |     """Verify a Schnorr signature (see BIP 340).
```

file_excerpt · vendor/embit/src/embit/util/key.py · line 572 · SHA-256 f0e6c6749e812b18... · File 0850b1637dafc13c...

```
571 |
572 | def sign_schnorr(key, msg, aux=None, flip_p=False, flip_r=False):
573 |     """Create a Schnorr signature (see BIP 340)."""
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Classify each usage, record data lifetimes, and plan migration to a post-quantum algorithm; introduce an abstraction layer for crypto-agility.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11971 · CODE-ECB-MODE

Info

ECB block-cipher mode used in src/krux/krux_settings.py

Potential cryptographic risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-29): not applicable to this code.** Not a use of the mode. src/krux/krux_settings.py:425 defines MODE_NAMES = {ucryptolib.MODE_ECB: "AES-ECB", ...}, a mapping from mode constant to the label shown in the settings menu, so the device can print which format a user picked. No key, no plaintext and no call to ucryptolib.aes appears here. The design decision this finding is really about is in src/krux/kef.py and is decided separately below; counting it twice would report one choice as two problems.
>
> automated severity: High.

1 usage(s) of ECB mode (first at line 426).

Affected asset	file:src/krux/krux_settings.py
Data source	src/krux/krux_settings.py
Detection method	Regular-expression matching of language-specific API patterns.
Analyzer	source_code 1.3.0 · 2026-11-04 18:52 UTC
Priority score	30.6 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable

POTENTIAL IMPACT

ECB reveals patterns in encrypted data and provides no integrity.

REPRODUCTION STEPS

```
Open line 426 in the supplied file src/krux/krux_settings.py and inspect the quoted lines (see evidence).
```

EVIDENCE

file_excerpt · src/krux/krux_settings.py · line 426 · SHA-256 ed4728795604753e... · File 70dd0a40cb8768fd...

```
424 |     # defined in krux.encryption.VERSIONS
425 |     MODE_NAMES = {
426 |         ucryptolib.MODE_ECB: "AES-ECB",
427 |         ucryptolib.MODE_CBC: "AES-CBC",
428 |         ucryptolib.MODE_CTR: "AES-CTR",
```

LIMITATIONS OF THE CHECK

Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Use an AEAD mode (GCM) with unique nonces.

RETEST PROCEDURE

Re-scan the source and review the call sites listed in the evidence.

NIST SP 800-131A Rev. 2 — Transitioning the Use of Cryptographic Algorithms and Key Lengths

QSI-2026-11976 · CODE-WEAK-PBKDF2

Info

Low PBKDF2 iteration count (2500) in vendor/embit/src/embit/slip39.py

Potential cryptographic risk | Confidence: Medium

Analyst review (QSI review, 2026-10-29): not applicable to this code. The iteration count is a protocol constant, not a hardening parameter. vendor/embit/src/embit/slip39.py:51 calls hashlib.pbkdf2_hmac("sha256", i + passphrase, salt + right, 2500 << exponent, half) inside _crypt, which is SLIP-39's four-round Feistel construction: 2500 per round times four rounds is the specification's base iteration count of 10000, and `exponent` is SLIP-39's own iteration-exponent field, which a share carries so that any implementation can reproduce it. Raising it to OWASP's 600,000 — the rule's recommendation, which is correct guidance for password storage — would produce shares that no other SLIP-39 implementation could read, and would therefore destroy the interoperability that makes a backup standard worth using. The rule is

> right that 2500 is low for a password; it is the wrong question for a protocol constant. Worth separating from a real concern: SLIP-39's security rests on the share secret, not on the passphrase being hard to guess, so the iteration count is not what protects a strong share.
> automated severity: Medium.

PBKDF2 with 2500 iterations at line 51.

Affected asset	file:vendor/embit/src/embit/slip39.py
Data source	vendor/embit/src/embit/slip39.py
Detection method	Regular-expression matching of language-specific API patterns.
Analyzer	source_code 1.3.0 · 2026-11-04 18:52 UTC
Priority score	30.6 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable

POTENTIAL IMPACT

Low iteration counts make offline password guessing cheaper.

REPRODUCTION STEPS

```
Open line 51 in the supplied file vendor/embit/src/embit/slip39.py and inspect the quoted lines (see evidence).
```

EVIDENCE

file_excerpt · vendor/embit/src/embit/slip39.py · line 51 · SHA-256 e0acab3c3d0997f0… · File 363e0dbd9282b7d6…

```
49 |     salt = b"shamir" + id.to_bytes(2, "big")
50 |     for i in indices:
51 |         f = hashlib.pbkdf2_hmac(
52 |             "sha256",
53 |             i + passphrase,
```

LIMITATIONS OF THE CHECK

Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Follow current OWASP Password Storage guidance (e.g. 600,000 iterations for PBKDF2-HMAC-SHA256) or use Argon2id.

RETEST PROCEDURE

Re-scan the source and review the call sites listed in the evidence.

NIST SP 800-132 — Recommendation for Password-Based Key Derivation

QSI-2026-11979 · DEP-ADVISORY-DEV

Info

40 advisories in 8 development or build-only PyPI package(s)

Improvement recommendation | Confidence: Medium

> **Analyst review (QSI review, 2026-10-29): not applicable to this code.** Correct as a single non-runtime summary, and it is the one thing in this audit that the submodule fix made worse before a second fix made it right. vendor/embit/poetry.lock pins 55 packages and every one

> of them is `optional = true` behind `markers = 'extra == "dev"'`, so embit has no unconditional runtime dependencies at all: the lock file describes how embit is developed and documented, not what it ships. Read by Poetry groups alone they all appeared to be runtime — Poetry 2 files extras under the main group — and this repository briefly reported one critical and sixteen high advisories, eleven of them virtualenv, for a device that runs MicroPython on a K210 and pip-installs nothing. The parser now treats an optional package gated behind a development-shaped extra as non-runtime. Nothing in this group reaches the firmware.

OSV.dev lists 40 advisories (17 rated high or critical) for 8 PyPI package(s) that only development or build tooling pulls in (vendor/embit/poetry.lock): e.g. black@26.3.1 [dev]: GHSA-cg8m-r9f2-5wm2; certifi@2026.2.25 [dev]: GHSA-6q48-6c7w-9gw2; filelock@3.25.2 [dev]: GHSA-567q-5qvw-pf92, GHSA-69hj-f68j-2879, GHSA-8mx5-7m5m-r77g, GHSA-9vhv-jxww-xfm5, GHSA-cwqr-hh33-gh44, GHSA-fq56-3cx3-fjxp, GHSA-j8f7-rjxc-mr56, GHSA-rfpc-qcw5-3pc4, GHSA-v4x8-3wr5-3wq8; idna@3.11 [dev]: GHSA-65pc-fj4g-8rjx; mkdocs-material@9.7.6 [dev]: GHSA-xvg9-69gf-fjrf. The manifests mark them as such (devDependencies, "dev": true in package-lock.json, pnpm importers sections, [dev-dependencies]/[build-dependencies] reached through Cargo.lock, or — for yarn.lock, which records no scope at all — package.json reached through the lock graph), so they are not part of what ships.

Affected asset	dependencies:PyPI:non-runtime
Data source	OSV.dev
Detection method	OSV.dev querybatch lookup of exact versions; scope from devDependencies, package-lock.json dev flags, pnpm importers sections, the Cargo.lock graph rooted at the Cargo.toml tables, and — for yarn.lock, which records no scope — the lock graph walked from package.json. Where a runtime requirement cannot be resolved in a lock, that lock is left unclassified rather than risking a package being set aside wrongly.
Analyzer	dependencies 1.3.0 · 2026-11-04 18:52 UTC
Priority score	18.3 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable

POTENTIAL IMPACT

They do not reach users of the product. They can affect developer machines and CI (supply chain): a compromised or vulnerable tool can tamper with builds.

REPRODUCTION STEPS

```
Look the packages listed in the evidence up at https://osv.dev and check their scope in vendor/embit/poetry.lock.
```

EVIDENCE

tool_output · OSV.dev (PyPI) · vendor/embit/poetry.lock · SHA-256 47704e2f28f1e9b2...

```
black@26.3.1 [dev]: GHSA-cg8m-r9f2-5wm2
certifi@2026.2.25 [dev]: GHSA-6q48-6c7w-9gw2
filelock@3.25.2 [dev]: GHSA-567q-5qvw-pf92, GHSA-69hj-f68j-2879, GHSA-8mx5-7m5m-r77g, GHSA-9vhv-jxww-xfm5, GHSA-cwqr-hh33-gh44, GHSA-fq56-3cx3-fjxp, GHSA-j8f7-rjxc-mr56, GHSA-rfpc-qcw5-3pc4, GHSA-v4x8-3wr5-3wq8
idna@3.11 [dev]: GHSA-65pc-fj4g-8rjx
mkdocs-material@9.7.6 [dev]: GHSA-xvg9-69gf-fjrf
pymdown-extensions@10.21 [dev]: GHSA-62q4-447f-wv8h, GHSA-9xwg-3r6f-jcx2, GHSA-gm37-52c6-37mw
urllib3@2.6.3 [dev]: GHSA-8988-9cw3-xx77, GHSA-gh4c-6fx4-qh6g, GHSA-mf9v-mfxr-j63j, GHSA-qccp-gfcp-xxvc, GHSA-vxq7-64xx-v4gw
virtualenv@21.2.0 [dev]: GHSA-394h-qwmc-48pf, GHSA-43mp-35q3-x3fq, GHSA-494p-63j8-28p3,
```

```
GHSA-558c-9hv2-4prj, GHSA-5vjq-rrrf-7h2q, GHSA-6w3r-xr62-4xcp, GHSA-8rjx-v5ww-45pp, GHSA-94p9-xgh2-
xp45, GHSA-9h9j-4vrj-gf7g, GHSA-c947-3pg5-gm8q, GHSA-c978-59m8-c77v, GHSA-cw65-gx24-rc82, GHSA-
cx28-75q4-x26q, GHSA-fc2g-4w5p-6whm, GHSA-hhxq-47xx-j5xx, GHSA-p58f-9548-mpm2, GHSA-qmx5-4p3x-wvqf,
GHSA-vxj4-q4g9-crj7, GHSA-x78j-v8h9-3j2q
```

LIMITATIONS OF THE CHECK

Advisory data comes from OSV.dev at the recorded time. Scopes come from the manifests; a registry crate's own build dependencies cannot be told apart in Cargo.lock, so the runtime set may be larger than reality, never smaller.

RECOMMENDATION

Update the tooling during routine maintenance; prioritise packages that run in CI with secrets or that produce release artefacts.

RETEST PROCEDURE

Re-run the dependency scan after updating.

OSV.dev — open-source vulnerability database and API

Evidence

ID	Type	Data source	SHA-256
QSI-2026-11969	file_excerpt	src/krux/kef.py (line 30)	983f2cc48989ecc4a9f6c09815c088adaee8547eabd249ae4291e761a
QSI-2026-11969	file_excerpt	src/krux/kef.py (line 38)	e840c4c38d9dd0654dfd9922ecb81f4c117f1e3137ec29e256a5d470e
QSI-2026-11969	file_excerpt	src/krux/kef.py (line 50)	4102c93b1240502843f64f9563a027599f489a8e989673c7958b2c3b9
QSI-2026-11969	file_excerpt	src/krux/kef.py (line 56)	2b53d9a1ca1d33e5ab513ca4295be4bc013ab6569b4107406722600a5
QSI-2026-11969	file_excerpt	src/krux/kef.py (line 63)	dbd2e22444870467bd8df2254c9813892fb16fa3782ccda465b67e7dc
QSI-2026-11978	file_excerpt	vendor/embit/src/embit/util/key.py (line 374)	6a16bf61b745add23af0bcc22cb0758b1719a6aead17b725bb75597ba
QSI-2026-11972	file_excerpt	vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_parsing.c (line 11)	9197354c850cb53bbc475b348c0f97bb3f550706fcced8bf483336d62
QSI-2026-11972	file_excerpt	vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_parsing.c (line 19)	234aeaaf6b28e1d2ff43854e60a3119e3c3f55229e18841e2dc766404
QSI-2026-11972	file_excerpt	vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_parsing.c (line 140)	75ad5d0cba25ca64adb6d971da937879704b4abc8845c17dc1da9fa1e
QSI-2026-11972	file_excerpt	vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_parsing.h (line 57)	25af7e242d92af0d3c46ae8d8a48030e247c4bfdeba65068030e64eb4
QSI-2026-11972	file_excerpt	vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_parsing.h (line 59)	4c35a81e99a7eaf5ea7315346f3b8cacb0832f7c4a3b1a3d003ff0bd3
QSI-2026-11972	file_excerpt	vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_parsing.h (line 88)	85efcade66268dd4d0b42e8f31b634ea4cd93d9d55cdc2e036715108d
QSI-2026-11972	file_excerpt	vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_privatekey_parsing.h (line 34)	163cac24f6b60e798317e2383fb603da718e728035262dc9759d7cf3f
QSI-2026-11972	file_excerpt	vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_privatekey_parsing.h (line 36)	2741d1869779ddac4f151ac1740c338ba299b2bed7b98a38e6042acb6
QSI-2026-11972	file_excerpt	vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1.h (line 85)	976a23a4a733f368307a87186feac41d258d0c7eae4fef7da5eddec53
QSI-2026-11972	file_excerpt	vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1.h (line 417)	15748a6fd21f774cca6b0c463fb56ef156b0a05c17cc86d0a52c76c0e

QSI-2026-11974	file_excerpt	vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_schnorrsig.h (line 82)	44a2365db81547948644f36130e817fe690091caa187560ab468f3458
QSI-2026-11974	file_excerpt	vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_schnorrsig.h (line 100)	72dc7958413515e7228437fe19dec8a24a240083981c855296f98cdf0
QSI-2026-11974	file_excerpt	vendor/embit/secp256k1/secp256k1-zkp/src/modules/musig/example.c (line 160)	e6ab7181c8dbb6504611247b2678ae83985acb1cf70e10429645f5d86
QSI-2026-11974	file_excerpt	vendor/embit/secp256k1/secp256k1-zkp/src/modules/musig/main_impl.h (line 460)	8347c1cd3e521a8ba5e1c0b465436477efc94e0749e2b6314e1928dcf
QSI-2026-11974	file_excerpt	vendor/embit/secp256k1/secp256k1-zkp/src/modules/schnorrsig/main_impl.h (line 98)	e73ca8d800b70a374cf5e3972e2d153290a93d2d12334ada5196282ce
QSI-2026-11974	file_excerpt	vendor/embit/secp256k1/secp256k1-zkp/src/modules/schnorrsig/main_impl.h (line 111)	0dd4fa6557933bdf32a316d69c6f05a1a458a5fd3f056f03a250c6cee
QSI-2026-11974	file_excerpt	vendor/embit/secp256k1/secp256k1-zkp/src/modules/schnorrsig/main_impl.h (line 117)	d433a64049d5b15ad012fcc92d25efe3f9c0faef114da2cfbe07f26c8
QSI-2026-11974	file_excerpt	vendor/embit/src/embit/util/ctypes_secp256k1.py (line 287)	57e08893e8b0d02ff62160e83bf0ee2a5297829ebdd67694ce5df8eac
QSI-2026-11974	file_excerpt	vendor/embit/src/embit/util/ctypes_secp256k1.py (line 293)	081fdf562b7a5fc864bee913ad182153c51fe01d652d42c0ccaec47ab
QSI-2026-11974	file_excerpt	vendor/embit/src/embit/util/ctypes_secp256k1.py (line 295)	ca6ef88f4a54753ab4313ab865dab0bd59f732d4276c85eb8d3f4d7a3
QSI-2026-11975	file_excerpt	vendor/embit/src/embit/liquid/pset.py (line 101)	b66b1e583d90306b059a9fd44ead20cf204eaa731e5f490ea219c83b4
QSI-2026-11975	file_excerpt	vendor/embit/src/embit/liquid/pset.py (line 102)	06f441e13a3c4ee80a5e7bc1c367c410548b47089185765cf11a4af02
QSI-2026-11975	file_excerpt	vendor/embit/src/embit/liquid/pset.py (line 300)	e13d6f4ff6fe3bd7adcceae1d254128bfd231d1754548bed4173819fc
QSI-2026-11975	file_excerpt	vendor/embit/src/embit/liquid/transaction.py (line 90)	ffe62aaa23ace0fd2936ab3f4a57b541efac76f5ce1fdc493f53d8712

QSI-2026-11975	file_excerpt	vendor/embit/src/embit/util/ctypes_secp256k1.py (line 949)	b7bf98d515fa733b6fa50b40868407081e5d056aea4aa1f8521535f39
QSI-2026-11975	file_excerpt	vendor/embit/src/embit/util/ctypes_secp256k1.py (line 960)	ae7600847f2badfdc520dc9b43caa54fe5c140fbc3f2618b08e27f378
QSI-2026-11975	file_excerpt	vendor/embit/src/embit/util/ctypes_secp256k1.py (line 971)	d061be5d1ea86ffb8675c67e3f0d729a08048a4c9b7bd6c3790b187b4
QSI-2026-11975	file_excerpt	vendor/embit/src/embit/util/secp256k1.py (line 51)	4485c26fa3e644b515ed1a6abf59b2d50f2bc77722ad56b84c581a20e
QSI-2026-11975	file_excerpt	vendor/embit/src/embit/util/secp256k1.py (line 52)	9ee2ecc76d93cb9f14a224de7eee8a4eb5a1dab003a01063faae76fc6
QSI-2026-11975	file_excerpt	vendor/embit/src/embit/util/secp256k1.py (line 53)	b419e6b04ad1f224679bd93af0189ee4ac479a9d15402a9c1c3e8a73e
QSI-2026-11970	file_excerpt	src/krux/key.py (line 252)	3571a45d6fb4255051dccedca3e72ab1bec1576060e8bce5a1d7dbe64
QSI-2026-11970	file_excerpt	src/krux/key.py (line 255)	4b287db1f3538b56bb615d288b4011bc3684c9fbcaae6603726b28e46
QSI-2026-11970	file_excerpt	src/krux/key.py (line 261)	70e842829bb3f2206eb1bc448bf9ec0a6a406c0ba40eff0460436711
QSI-2026-11970	file_excerpt	vendor/embit/src/embit/ec.py (line 3)	2da5ac4f0aac6b95a77ebbf62d29881b49cb5b77b6b6b0e545c568d12
QSI-2026-11970	file_excerpt	vendor/embit/src/embit/ec.py (line 18)	22af8149d1de86bdaa2f37e9a1ee21d18db2b64fbb9fcd55610a17ea2
QSI-2026-11970	file_excerpt	vendor/embit/src/embit/ec.py (line 24)	91eebf765cedecb0508205d819b8bdf939b5d9534e0cb94aab17156c7
QSI-2026-11970	file_excerpt	vendor/embit/src/embit/misc.py (line 7)	05b74d409d6158636eb6487a7526851bab9bc6f71ec1c6983c7c03cd2
QSI-2026-11970	file_excerpt	vendor/embit/src/embit/misc.py (line 9)	ff45e726cacf161b9a430429e3241edb8cf6b0a3b38b7970dee8086c4
QSI-2026-11970	file_excerpt	vendor/embit/src/embit/util/__init__.py (line 1)	00860f29eecbf17fef97f8df024ac6a0e1c1e9cbbe9369e049e938eac
QSI-2026-11970	file_excerpt	vendor/embit/src/embit/util/key.py (line 3)	fb27cc95d202f1f9be5c94d5caf6544118c775bffd007789a844330fc
QSI-2026-11973	file_excerpt	vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_ecdh.h (line 49)	72841f4795f99327f8d386d48f812972ad9b9c93c2aaa26a78a395894
QSI-2026-11973	file_excerpt	vendor/embit/secp256k1/secp256k1-zkp/src/modules/ecdh/main_impl.h (line 10)	65d8983e01c7b44d6401a1272abfba2e129f3af0eaaa88497c0a87143
QSI-2026-11973	file_excerpt	vendor/embit/secp256k1/secp256k1-zkp/src/modules/ecdh/main_impl.h (line 29)	99bef9ddb51d26be8ad1c573044618a7a1801ccdcb8c468cbce745d4c

QSI-2026-11973	file_excerpt	vendor/embit/src/embit/util/ctypes_secp256k1.py (line 131)	25ed92dcf661ed2360a198512a0006c9e51c0e8a86d6e1e6cb0b64dc8
QSI-2026-11973	file_excerpt	vendor/embit/src/embit/util/ctypes_secp256k1.py (line 265)	de340041c8677f93a6eb25e7cd23f166d2c5a072a02c006e4f7b86109
QSI-2026-11973	file_excerpt	vendor/embit/src/embit/util/ctypes_secp256k1.py (line 273)	9d8302f1aac7f738489ca0a9b1da65674b762df2abcb1df8bf0ba178b
QSI-2026-11973	file_excerpt	vendor/embit/src/embit/util/secp256k1.py (line 31)	4be965210b8e21d1911d60c3f2f6fcb51b6f14257125a3509ed6f700a
QSI-2026-11977	file_excerpt	vendor/embit/src/embit/util/key.py (line 535)	0597a48161191ebf461aaedda31f2a87a429eaaa13531cee7adae7137
QSI-2026-11977	file_excerpt	vendor/embit/src/embit/util/key.py (line 572)	f0e6c6749e812b18bfc62efa05381551783c523260a6d2f23ec1e1f80
QSI-2026-11971	file_excerpt	src/krux/krux_settings.py (line 426)	ed4728795604753e57fcefec111b007f9f44c8bdc96aba7efd7e3380e
QSI-2026-11976	file_excerpt	vendor/embit/src/embit/slip39.py (line 51)	e0acab3c3d0997f0fb4255d3150bb54b2560dd0c4f85952ae27123e58
QSI-2026-11979	tool_output	OSV.dev (PyPI) (vendor/embit/poetry.lock)	47704e2f28f1e9b2997e7f58043e0a0637c52977b48edc36f9c64c07f

Post-quantum assessment

	Key establishment (x = how long data must stay confidential; harvest now, decrypt later)	Signatures (x = how long signatures/ keys must stay trusted)
Data/key lifetime x	10	10
Migration time y	5	5
Assumed years to CRQC z	8 (early)	8
x + y − z	7 → migration would finish too late under this scenario	7 → migration would finish too late under this scenario

Planning assumptions: x for key establishment, x for signatures, y are the defaults of the risk profile “Crypto exchange, custodian or wallet”, not the client's figures; with the client's actual lifetimes the verdict can change.

Post-quantum maturity: Level 0 of 4 — no post-quantum cryptography

- Classical only — threshold signing and MPC protocols: Pedersen commitment (4 file(s))

Next step: Inventory where classical public-key cryptography protects long-lived data and keys, and choose the first migration: usually a hybrid TLS key exchange and wrapped keys.

Levels: 0 none; 1 present, not wired; 2 in use outside the critical paths; 3 hybrid in a critical path; 4 post-quantum by default in the critical paths (root of trust, key wrapping, channel key exchange). Judged from code and manifests.

What the quantum-vulnerable algorithms do

Role	Algorithms (files)	ID	First step
threshold signing and MPC protocols	Pedersen commitment (4)	QSI-2026-11975	Track post-quantum threshold signatures (NIST IR 8214C) and keep the protocol layer ready to swap the signature scheme; record which keys and shares must stay valid for how long
transaction and message signing	ECDSA (31), Schnorr (BIP-340) (7)	QSI-2026-11972, QSI-2026-11974	Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains adopt them
role not determined	ECDSA (5), ECDH (4), Schnorr (BIP-340) (1)	QSI-2026-11970, QSI-2026-11973, QSI-2026-11977	Classify each usage, record data lifetimes, and plan migration to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it; introduce an abstraction layer for crypto-agility

Roles are judged from file paths, function names and the code around each match; review confirms them. Root-of-trust keys and wrapped keys come first: they are hard to replace or can be recorded today.

z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

Quantum status	Count
weak classically	2
reduced margin (Grover), approved	10
Grover: acceptable to NIST, below ECCG/ANSSI guidance	17
n/a	5
vulnerable (Shor)	59

Detailed findings: QSI-2026-11972, QSI-2026-11974, QSI-2026-11975, QSI-2026-11970, QSI-2026-11973, QSI-2026-11977

PQC readiness questionnaire

No readiness questionnaire was completed for this project.

Quantum algorithms relevant to this inventory

Quantum algorithm	Effect	Inventoried algorithms (rows)	What it means for this audit
Shor's algorithm — elliptic-curve discrete logarithms Q1: quantum computation offline, classical access to the victim	breaks	ECDH (4), ECDSA (43), Pedersen commitment (4), Schnorr (BIP-340) (8)	Elliptic-curve keys are expected to fall before RSA keys of comparable classical strength. On blockchains, coins whose public key is already on chain (reused addresses, pay-to-public-key and Taproot outputs) are exposed first; plan the move to post-quantum signatures and avoid key reuse in the meantime.
Grover's algorithm — key search and preimages Q1: quantum computation offline, classical access to the victim	weakens (larger keys restore the margin)	HMAC (6), PBKDF2 (4), RIPEMD-160 (2), SHA-2 (17)	Not a break. 256-bit keys (AES-256, ChaCha20) keep a comfortable margin; AES-128 meets NIST category 1, while some authorities ask for 256-bit keys in quantum-resistant contexts (see the AES-128 entry). Prefer 256-bit keys for long-lived data; 256-bit hashes need no change for preimage resistance.
Brassard–Høyer–Tapp — collision search Q1: quantum computation offline, classical access to the victim	weakens (larger keys restore the margin)	RIPEMD-160 (2), SHA-2 (17)	256-bit hashes (SHA-256, SHA3-256) remain adequate for collision resistance; 160-bit hashes are already weak classically (SHA-1 collisions are practical). Use 384/512-bit hashes where NIST categories 4–5 are required.

Simon's algorithm — superposition attacks on symmetric constructions Q2: superposition queries to the victim's keyed primitive	only with superposition queries	GCM mode (2)	No action for ordinary deployments of AES-GCM or CBC-MAC — the attack model does not apply. Record it as a model assumption; see the offline variant for Even–Mansour-type designs.

Effects: breaks — replace the algorithm; weakens — larger keys or outputs restore the margin; included in standard parameters — no action at standard parameter sets; only with superposition queries — not applicable to ordinary deployments.

Limitations and assumptions

- This report presents automated and documented analysis results for the stated scope. It is not a legal opinion, a certification or an attestation of compliance with any standard.
- z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.
- tls_key_exchange_groups — Insufficient data: 56 (examples: vendor/embit/poetry.lock line 9: not in the algorithm registry, not assessed: ["main"]; vendor/embit/poetry.lock line 25: not in the algorithm registry, not assessed: ["main"]; vendor/embit/poetry.lock line 46: not in the algorithm registry, not assessed: ["main"])
- osv_advisories — Insufficient data: 1 (examples: PyPI/virtualenv@21.2.0: 23 advisories at OSV.dev, the first 20 are listed as findings; see OSV.dev for the rest)

Remediation plan

QSI-2026-11969 · P1 · 30 days root-cause fix

Use an AEAD mode (GCM) with unique nonces.

Rationale	ECB reveals patterns in encrypted data and provides no integrity.
Expected outcome	Re-scan the source and review the call sites listed in the evidence.
Complexity	medium
Tests before change	Open line 30 in the supplied file src/krux/kef.py and inspect the quoted lines (see evidence).
Tests after change	Re-scan the source and review the call sites listed in the evidence.
Closure criteria	Re-scan the source and review the call sites listed in the evidence.

QSI-2026-11978 · P1 · 30 days root-cause fix

Use the platform CSPRNG (secrets, os.urandom, crypto.getRandomValues, crypto/rand, SecureRandom, OsRng) or a hardware TRNG for every key, seed and mnemonic.

Rationale	Mnemonics, seeds or private keys drawn from such a generator can be enumerated from its small internal state: Milk Sad (CVE-2023-39910, mt19937 behind 'bx seed') and Trust Wallet (CVE-2023-31290) lost funds this way.
Expected outcome	Re-scan the source and review the call sites listed in the evidence.
Complexity	medium
Tests before change	Open line 374 in the supplied file vendor/embit/src/embit/util/key.py and inspect the quoted lines (see evidence).
Tests after change	Re-scan the source and review the call sites listed in the evidence.
Closure criteria	Re-scan the source and review the call sites listed in the evidence.

QSI-2026-11972 · P3 · 90 days short-term risk reduction

Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-11974 · P3 · 90 days short-term risk reduction

Add Schnorr (BIP-340) usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-11975 · P3 · 90 days short-term risk reduction

Add Pedersen commitment usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-11970 · P3 · 90 days short-term risk reduction

Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-11973 · P3 · 90 days

short-term risk reduction

Add ECDH usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-11977 · P3 · 90 days

short-term risk reduction

Add Schnorr (BIP-340) usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-11972 · P4 · 90 days

root-cause fix

Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it or a hybrid scheme in use.

QSI-2026-11974 · P4 · 90 days root-cause fix

Plan migration of Schnorr (BIP-340) to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows or a hybrid scheme in use.

QSI-2026-11975 · P4 · 90 days root-cause fix

Plan migration of Pedersen commitment to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows or a hybrid scheme in use.

QSI-2026-11970 · P4 · 90 days root-cause fix

Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it or a hybrid scheme in use.

QSI-2026-11973 · P4 · 90 days root-cause fix

Plan migration of ECDH to Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH or a hybrid scheme in use.

QSI-2026-11977 · P4 · 90 days root-cause fix

Plan migration of Schnorr (BIP-340) to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows or a hybrid scheme in use.

QSI-2026-11971 · P4 · 90 days root-cause fix

Use an AEAD mode (GCM) with unique nonces.

Rationale	ECB reveals patterns in encrypted data and provides no integrity.
Expected outcome	Re-scan the source and review the call sites listed in the evidence.
Complexity	medium
Tests before change	Open line 426 in the supplied file src/krux/krux_settings.py and inspect the quoted lines (see evidence).
Tests after change	Re-scan the source and review the call sites listed in the evidence.
Closure criteria	Re-scan the source and review the call sites listed in the evidence.

QSI-2026-11976 · P4 · 90 days root-cause fix

Follow current OWASP Password Storage guidance (e.g. 600,000 iterations for PBKDF2-HMAC-SHA256) or use Argon2id.

Rationale	Low iteration counts make offline password guessing cheaper.
Expected outcome	Re-scan the source and review the call sites listed in the evidence.
Complexity	medium
Tests before change	Open line 51 in the supplied file vendor/embit/src/embit/slip39.py and inspect the quoted lines (see evidence).
Tests after change	Re-scan the source and review the call sites listed in the evidence.
Closure criteria	Re-scan the source and review the call sites listed in the evidence.

QSI-2026-11979 · P4 · 90 days root-cause fix

Update the development and build dependencies listed in the evidence.

Rationale	Eliminates the root cause described in the finding.
Expected outcome	The weakness is no longer present on re-test.
Complexity	low
Dependencies	Change window and owner approval.
Side effects	Tooling upgrades may change build output; compare release artefacts.
Tests before change	Reproduce the finding using the documented steps.
Tests after change	Re-run the retest procedure from the finding.
Closure criteria	The dependency scan reports no advisories for tooling packages, or the remaining ones are recorded.

Priorities for 30, 60 and 90 days

30 days

Priority	ID	Action	Type
P1	QSI-2026-11969	Use an AEAD mode (GCM) with unique nonces.	root-cause fix
P1	QSI-2026-11978	Use the platform CSPRNG (secrets, os.urandom, crypto.getRandomValues, crypto/rand, SecureRandom, OsRng) or a hardware TRNG for every key, seed and mnemonic.	root-cause fix

60 days

Priority	ID	Action	Type
None			

90 days

Priority	ID	Action	Type
P3	QSI-2026-11972	Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-11974	Add Schnorr (BIP-340) usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-11975	Add Pedersen commitment usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-11970	Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-11973	Add ECDH usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-11977	Add Schnorr (BIP-340) usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P4	QSI-2026-11972	Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-11974	Plan migration of Schnorr (BIP-340) to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-11975	Plan migration of Pedersen commitment to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix

P4	QSI-2026-11970	Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-11973	Plan migration of ECDH to Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-11977	Plan migration of Schnorr (BIP-340) to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-11971	Use an AEAD mode (GCM) with unique nonces.	root-cause fix
P4	QSI-2026-11976	Follow current OWASP Password Storage guidance (e.g. 600,000 iterations for PBKDF2-HMAC-SHA256) or use Argon2id.	root-cause fix
P4	QSI-2026-11979	Update the development and build dependencies listed in the evidence.	root-cause fix

Verification after remediation

ID	Retest procedure
QSI-2026-11969	Re-scan the source and review the call sites listed in the evidence.
QSI-2026-11978	Re-scan the source and review the call sites listed in the evidence.
QSI-2026-11972	Re-run the inventory after refactoring.
QSI-2026-11974	Re-run the inventory after refactoring.
QSI-2026-11975	Re-run the inventory after refactoring.
QSI-2026-11970	Re-run the inventory after refactoring.
QSI-2026-11973	Re-run the inventory after refactoring.
QSI-2026-11977	Re-run the inventory after refactoring.
QSI-2026-11971	Re-scan the source and review the call sites listed in the evidence.
QSI-2026-11976	Re-scan the source and review the call sites listed in the evidence.
QSI-2026-11979	Re-run the dependency scan after updating.

Tools used

Tool	Version
blockchain	1.0.0
certificates	1.2.0
chain_map	1.0.0
configs	1.1.0
dependencies	1.3.0
inventory_document	1.1.0
solidity	1.3.0
source_code	1.4.0
tls_code	1.1.0
tls_live	1.0.0
qsi-engine	1.3.0
OSV.dev API	v1

Sources and normative documents

Reviewed: 2026-10-13

1. A. Chailloux, M. Naya-Plasencia, A. Schrottenloher — An Efficient Quantum Collision Search Algorithm and Implications on Symmetric Cryptography (ASIACRYPT 2017; IACR ePrint 2017/847) — IACR ePrint. https://eprint.iacr.org/2017/847 (peer-reviewed article: Õ(2^{2n/5}) time, O(n) qubits, Õ(2^{n/5}) classical memory)
2. C. Chevignard, P.-A. Fouque, A. Schrottenloher — Reducing the Number of Qubits in Quantum Discrete Logarithms on Elliptic Curves (IACR ePrint 2026/280) — IACR ePrint. https://eprint.iacr.org/2026/280 (revision of a EUROCRYPT 2026 paper: 3.12n + o(n) qubits; for n = 256, 1193 qubits and 22 runs of 2^38.98 Toffoli gates)
3. C. Zalka — Grover's quantum searching algorithm is optimal (Phys. Rev. A 60, 2746, 1999; arXiv:quant-ph/9711070) — APS. https://doi.org/10.1103/PhysRevA.60.2746 (peer-reviewed article: optimal for any success probability; parallel search gains no more than splitting the space)
4. CVE-2023-31290 — Trust Wallet Core: 32 bits of mnemonic entropy (mt19937 seeded with one 32-bit value) — NIST NVD. https://nvd.nist.gov/vuln/detail/CVE-2023-31290 (published 2023-04-27; Trust Wallet Core before 3.1.1 (browser extension 0.0.172-0.0.182), exploited in the wild in December 2022 and March 2023: only about four billion possible mnemonics)
5. CVE-2023-39910 — Libbitcoin Explorer 'bx seed' entropy (Milk Sad): mt19937 limits it to 32 bits — NIST NVD. https://nvd.nist.gov/vuln/detail/CVE-2023-39910 (published 2023-08-09; Libbitcoin Explorer 3.0.0-3.6.0, exploited in the wild in 2023)
6. CWE-338 — Use of Cryptographically Weak Pseudo-Random Number Generator (PRNG) — MITRE. https://cwe.mitre.org/data/definitions/338.html (weakness catalogue entry: the consequence is predictable values where unpredictability was assumed, which is what makes a token or key guessable)
7. Common Vulnerability Scoring System v3.1: Specification Document — FIRST. https://www.first.org/cvss/v3.1/specification-document (final)
8. D. J. Bernstein — Cost analysis of hash collisions: Will quantum computers make SHARCS obsolete? (2009) — cr.yp.to. https://cr.yp.to/hash/collisioncost-20090823.pdf (paper dated 2009-08-23)
9. D. Litinski — How to compute a 256-bit elliptic curve private key with only 50 million Toffoli gates (arXiv:2306.08585) — arXiv. https://arxiv.org/abs/2306.08585 (preprint (2023-06-14): 6000 modules × 1152 physical qubits, one key per 10 minutes under its architecture assumptions)
10. D. R. Simon — On the Power of Quantum Computation (SIAM J. Comput. 26, 1997) — SIAM. https://doi.org/10.1137/S0097539796298637 (peer-reviewed article)
11. FIPS 203 — Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM) — NIST. https://csrc.nist.gov/pubs/fips/203/final (final (August 2024))
12. FIPS 204 — Module-Lattice-Based Digital Signature Standard (ML-DSA) — NIST. https://csrc.nist.gov/pubs/fips/204/final (final (August 2024))
13. FIPS 205 — Stateless Hash-Based Digital Signature Standard (SLH-DSA) — NIST. https://csrc.nist.gov/pubs/fips/205/final (final (August 2024))
14. G. Brassard, P. Høyer, A. Tapp — Quantum Algorithm for the Collision Problem (LATIN '98; arXiv:quant-ph/9705002) — Springer / arXiv. https://arxiv.org/abs/quant-ph/9705002 (peer-reviewed article: collisions of r-to-one functions in O((N/r)^{1/3}) expected evaluations)
15. H. Kuwakado, M. Morii — Quantum distinguisher between the 3-round Feistel cipher and the random permutation (IEEE ISIT 2010) — IEEE. https://ieeexplore.ieee.org/document/5513654 (peer-reviewed conference paper)
16. J. Proos, C. Zalka — Shor's discrete logarithm quantum algorithm for elliptic curves (Quantum Inf. Comput. 3(4), 2003; arXiv:quant-ph/0301141) — Quantum Information & Computation. https://doi.org/10.26421/QIC3.4-3 (peer-reviewed article)
17. L. K. Grover — A fast quantum mechanical algorithm for database search (STOC 1996) — ACM. https://doi.org/10.1145/237814.237866 (peer-reviewed article)
18. M. Amy et al. — Estimating the cost of generic quantum pre-image attacks on SHA-2 and SHA-3 (SAC 2016; arXiv:1603.09383) — Springer / arXiv. https://arxiv.org/abs/1603.09383 (peer-reviewed article: SHA-256 about 2^153.8 surface-code cycles on 2^12.6 logical qubits; SHA3-256 about 2^146.5 cycles on 2^20)
19. M. Cain et al. — Shor's algorithm is possible with as few as 10,000 reconfigurable atomic qubits (arXiv:2603.28627) — arXiv. https://arxiv.org/abs/2603.28627 (preprint (2026-03-30): P-256 in a few days with 26,000 physical neutral-atom qubits; RSA-2048 one to two orders of magnitude longer)
20. M. Kaplan, G. Leurent, A. Leverrier, M. Naya-Plasencia — Breaking Symmetric Cryptosystems using Quantum Period Finding (CRYPTO 2016; arXiv:1602.05973) — Springer / arXiv. https://arxiv.org/abs/1602.05973 (peer-reviewed article: superposition-query forgeries on CBC-MAC, PMAC, GMAC, GCM, OCB)
21. M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018) — IEEE. https://doi.org/10.1109/MSP.2018.3761723 (peer-reviewed article; source of the x + y > z migration inequality)

22. M. Roetteler, M. Naehrig, K. Svore, K. Lauter — Quantum resource estimates for computing elliptic curve discrete logarithms (ASIACRYPT 2017) — IACR / arXiv. https://arxiv.org/abs/1706.06752 (peer-reviewed article: at most 9n + 2⌈log2 n⌉ + 10 qubits and 448 n^3 log2 n + 4090 n^3 Toffoli gates)
23. NIST IR 8547 — Transition to Post-Quantum Cryptography Standards — NIST. https://csrc.nist.gov/pubs/ir/8547/final (final (2026-10-27; the initial public draft of November 2024 is obsoleted). Tables 2 and 4: ECDSA, EdDSA and RSA signatures, finite-field and elliptic-curve DH and MQV, and RSA key establishment are deprecated after 2030 and disallowed after 2035 at every key size — the draft deprecated only the 112-bit level. A composite (hybrid) key establishment stays FIPS 140-validatable while one input comes from an approved scheme. Repeats the federal deadlines of OMB M-26-15: post-quantum key establishment by 2030-12-31 and signatures by 2031-12-31 for high value assets and high impact systems)
24. NIST SP 800-131A Rev. 2 — Transitioning the Use of Cryptographic Algorithms and Key Lengths — NIST. https://csrc.nist.gov/pubs/sp/800/131/a/r2/final (final (2019))
25. NIST SP 800-132 — Recommendation for Password-Based Key Derivation — NIST. https://csrc.nist.gov/pubs/sp/800/132/final (final (2010))
26. NIST — Post-Quantum Cryptography Standardization: Evaluation Criteria, Security (Call for Proposals, Section 4.A.5) — NIST. https://csrc.nist.gov/Projects/Post-Quantum-Cryptography/Post-Quantum-Cryptography-Standardization/Evaluation-Criteria/Security-(Evaluation-Criteria) (security categories: AES-128 key search 2^170/MAXDEPTH quantum gates, AES-192 2^233/MAXDEPTH, AES-256 2^298/MAXDEPTH; MAXDEPTH 2^40–2^96)
27. OSV.dev — open-source vulnerability database and API — Open Source Security Foundation. https://osv.dev/ (the database QSI queries for each exactly pinned version; every advisory id in a finding resolves at https://osv.dev/vulnerability/<id>)
28. P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997) — SIAM. https://doi.org/10.1137/S0097539795293172 (peer-reviewed article)
29. R. Babbush et al. — Securing Elliptic Curve Cryptocurrencies against Quantum Vulnerabilities: Resource Estimates and Mitigations (arXiv:2603.28846) — arXiv. https://arxiv.org/abs/2603.28846 (preprint (2026-03-30, v2 2026-04-15): 256-bit ECDLP with < 1200 logical qubits and < 90 M Toffoli gates (or < 1450 and < 70 M); < 500,000 physical qubits at 10^-3 error, minutes)
30. S. Jaques, M. Naehrig, M. Roetteler, F. Virdia — Implementing Grover oracles for quantum key search on AES and LowMC (EUROCRYPT 2020; arXiv:1910.01700) — Springer / arXiv. https://arxiv.org/abs/1910.01700 (peer-reviewed article: lower cost estimates for the AES key searches that define the NIST categories)

Appendices

A. Cryptographic inventory

Algorithm	Parameter	Quantum status	Classical status	Count	Locations	Examples
RIPEMD-160	RIPEMD-160	weak classically	legacy	2	2	vendor/embit/src/embit/descriptor/miniscript.py:245; vendor/embit/src/embit/hashes.py:6
HMAC		reduced margin (Grover), approved	acceptable	6	6	vendor/embit/src/embit/bip32.py:53; vendor/embit/src/embit/bip85.py:26; vendor/embit/src/embit/liquid/blip32.py:11
PBKDF2		reduced margin (Grover), approved	acceptable	3	3	src/krux/encryption.py:70; vendor/embit/src/embit/bip39.py:81; vendor/embit/src/embit/slip39.py:51
PBKDF2	2500 iterations	reduced margin (Grover), approved	legacy	1	1	vendor/embit/src/embit/slip39.py:51
SHA-2	SHA-256	Grover: acceptable to NIST, below ECCG/ANSSI guidance	acceptable	17	17	firmware/Kboot/build/ktool.py:1176; firmware/Kboot/src/bootloader_hi/include/sha256.h; firmware/Kboot/src/bootloader_hi/sha256.c
CBC mode		n/a	acceptable	3	3	src/krux/encryption.py:77; src/krux/kef.py:71; src/krux/krux_settings.py:427
GCM mode		n/a	acceptable	2	2	src/krux/kef.py:109; src/krux/krux_settings.py:429
ECDH		vulnerable (Shor)	acceptable	4	4	vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_ecdh.h:49; vendor/embit/secp256k1/secp256k1-zkp/src/modules/ecdh/main_impl.h:10; vendor/embit/src/embit/util/ctypes_secp256k1.py:131
ECDSA	secp256k1	vulnerable (Shor)	acceptable	31	31	src/krux/key.py:252; vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_parsing.h:57; vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_privatekey_parsing.h:34

ECDSA	vulnerable (Shor)	acceptable	12	12	vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_parsing.c:11; vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_parsing.h:88; vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1.h:85
Pedersen commitment	vulnerable (Shor)	acceptable	4	4	vendor/embit/src/embit/liquid/pset.py:101; vendor/embit/src/embit/liquid/transaction.py:90; vendor/embit/src/embit/util/ctypes_secp256k1.py:949
Schnorr (BIP-340)	vulnerable (Shor)	acceptable	8	8	vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_schnorrsig.h:82; vendor/embit/secp256k1/secp256k1-zkp/src/modules/musig/example.c:160; vendor/embit/secp256k1/secp256k1-zkp/src/modules/musig/main_impl.h:460

Long lists are grouped here; the full per-file list is in the JSON report, signed with the same key.

B. JSON

qsi.report/1 · SHA-256 e013074c7b0eda8ff8256cc33f333800e595ae0c48705d84b95076950e74be8d

C. Input files

File	Size	SHA-256	Redactions
pyproject.toml	7270	4f53081e2daf3738eb1ef71229589f7f607558490ab8aa21d13546864235f28d	0
src/boot.py	5126	ed6dd67981fd1f63c25b8c07b1311f1cbeac1f922403c5c665c2eac6f8cc6574	0
src/krux/auto_shutdown.py	2856	9cfffddc948166eb37ac504492279c0ce68ad4d541c21cb5ccb3d9c8f49dc2e2	0
src/krux/baseconv.py	6175	4218468691f5c3be2e19c828cc6ed349e3ec477f4ec508f6647be5e4e276e472	0
src/krux/bbqr.py	7978	76a0f655f208a2b02dd5999e60795c140f31e065ce53efce96fc0e0416bb644e	0
src/krux/bip39.py	1883	0b86c027dc511730270af88bc74064e5f928b079468e556e39466fc21c04b05f	0
src/krux/buttons.py	4865	905bf1cbb512328a0c0b5c1e5efe2a4b5153bbc42cb2142ff72eb72d1ee904bd	0
src/krux/camera.py	13002	c713cbf864edc7a580fd9b25046c0cf4086ff5b50bcfdde23c6f9150ff2dc588	0
src/krux/context.py	2132	abda7f6810dc2e3f93b223326e987949ea4d22bb1ef81bf6a29e2ece2254d119	0
src/krux/display.py	21334	1e4ae96199b713a6678328e599f87cea50e2c493f0b26fbe72aad994a4b80d7c	0
src/krux/encryption.py	8360	9de6eb5e72c65b8e84d501f59326d23ff05e59a7847f045f25e3ec7fa95ec0d4	0
src/krux/firmware.py	15012	ff909ff9614bb23fd7500d5929a59a79579200f08e92ef49732744eb5928c5eb	0
src/krux/format.py	2954	748b715d23a6bbf16002ff22877afba7794fde3b5b4323791acaf0ab9f9ecf04	0
src/krux/i2c.py	1643	c18036b1cffb270c8566353473e61eb86e85af7fd145c4faf4305f381a480dd0	0

src/krux/input.py	13205	cfdabc6b75617b814d9ee66adfe2911c89268ab09ec07182ffa6cd564c23ffa6	0
src/krux/kboard.py	2517	edfd764d80ff8a6aa2ed485559b962046000f025bddcfc1232226fd01d26e9c3	0
src/krux/kef.py	19790	deb09c47bf20998696f11ed9b9fcc95bcb856b2def8428190c1044c1cf119a59	0
src/krux/key.py	12792	f9176b1f127dae2264bd8c084b341118c3b3ff8d789d29d3bb2081339c005f85	0
src/krux/krux_settings.py	17893	70dd0a40cb8768fd8a3b572ffe5ec1619a56858561efd4a94ffcf8ef4f9d453c	0
src/krux/light.py	2079	6098e9cd0468fb22abbd504e48bd2a0d21b30ab93eb38e94972be09cf57172b4	0
src/krux/metadata.py	1228	59201e56c54ad8aa1a418cd32d54d871c26041ce84f03625543ecb916360eafd	0
src/krux/pages/__init__.py	40646	7c2c51b227a5229a873ea69ecced11f645e2881c59dbb3024bbc26b9678a2671	0
src/krux/pages/capture_entropy.py	9406	cf4d595def6df6cf17c40c32cf283061722605af432c8322447b5812a1bc190e	0
src/krux/pages/datum_tool.py	30091	8011b6911b152f929ab04e0f5be01c5cd6e1477a511a6584ef4a1235c49fb5ef	0
src/krux/pages/disclaimer.py	5343	b931361060f361aa58f42e7eb03f0f4aecaaf32a4e81cccf1b1d82472ee65cdf	0
src/krux/pages/encryption_ui.py	28227	44ac4d6083386928b86f53c64b086abead14cc8377240c4781a15974e45fd73f	0
src/krux/pages/file_manager.py	8442	204ef3c46822679683d0506e3f443d7930684cd980a57ec88452ee9d88b795da	0
src/krux/pages/file_operations.py	5852	c4c212f195a601edbfa96d63a931fd5e9557604bb0523f1ccce5d9b9c4ae88cf	0
src/krux/pages/fill_flash.py	5157	5a9f1d5571eceb508f453f1dc443b7edc6eabb79146884de1033607be2752b98	0
src/krux/pages/flash_tools.py	12273	b539f22cd817332f5b8f7f2622dcb6c854fd405f8e37ef1e94e7a5e4dc1378f3	0
src/krux/pages/home_pages/__init__.py	1123	fe6beb85bc737f351cfb0031249e1c00f411fa722976ec2b5ba4e6dbdaefdc2a	0
src/krux/pages/home_pages/addresses.py	13882	dafe11805481132e269e11c9e76d63cfb396dcbeef2d34352026ec4b8f136448	0
src/krux/pages/home_pages/bip85.py	6156	45f983d3794d1e9f0f1a03f5888d5ccf99d2b3236b5f6004d2bba0321c897539	0
src/krux/pages/home_pages/home.py	18392	94b5d0c9a79b2e58fa1af91e4dcf3efc2bb50951d7676535fe6da595ae646781	0
src/krux/pages/home_pages/miniscript_indenter.py	7637	f89b326d6f84b99a4a195db4cdc16b353bcb4c5c67d31828dc2fbefd418d10a5	0
src/krux/pages/home_pages/mnemonic_backup.py	10419	f3f4acc818492f725a842bda4ab56118dc0999325c84e38ef0879a5a208130d6	0
src/krux/pages/home_pages/mnemonic_xor.py	7188	8cf5b6485b2fc6d95c7da4eca635f204148d7487669fc737e95020e515fcb348	0

src/krux/pages/home_pages/pub_key_view.py	5115	ff2d5750a184821cd9e03adc7dd65b482605550847d06c6f2409304bbbed64b7	0
src/krux/pages/home_pages/sign_message_ui.py	14457	72872068089939c3d0035ecd5d30b3355ec163000b338f68ab19af83b6861fd3	0
src/krux/pages/home_pages/wallet_descriptor.py	19232	06cbba09db29ce5d9b9d4a5d5cbd4ca4f6d204a96c4c683bee05616d0f6e4bb0	0
src/krux/pages/keypads.py	11940	499e7d0ccd87debdeeb9800195436a61d093696e1ac0acd72fab41a9911b0413	0
src/krux/pages/login.py	15936	8751e4fef68a6ccc7fc77576fde623fb727c1332869fdd549e3d8a22cf465889	0
src/krux/pages/mnemonic_editor.py	13796	abec17903df27f8be3ddab01d5c22d359196b6dea96643e087c3324159eca531	0
src/krux/pages/mnemonic_loader.py	17572	87c347f44333b4fb316f371622f38da0c953d4599ccc7dc362e11c69a88911e3	0
src/krux/pages/new_mnemonic/__init__.py	1123	fe6beb85bc737f351cfb0031249e1c00f411fa722976ec2b5ba4e6dbdaefdc2a	0
src/krux/pages/new_mnemonic/dice_rolls.py	13401	a30f5a5c6aaea2aa0ac0474bd5626109b869d4e7213a19219a64ff4a7e73d04b	0
src/krux/pages/print_page.py	3843	342853fdb1f3644852041e1267853fba9850385c25369cbadaf0724f7fbd9fbf	0
src/krux/pages/qr_capture.py	10880	15f93bb5f3cf36b2187ba5d81e8e5d245378b1fec62b8c7c1f2265d0353e9530	0
src/krux/pages/qr_view.py	22614	2d9a9a2005e73d0f6245644e470d7e9ee1c25acb38541c6e60b8a1f6196cc25b	0
src/krux/pages/screensaver.py	2524	b80cdd7734a2ae10e3952c8fe9531fc3be315c8ca9e7d9305497aa3fc1337fce	0
src/krux/pages/settings_page.py	20427	9e27f78fa83898cd3b692cd27ef05fb69382ca843955b9a6b622191ae2b93858	0
src/krux/pages/stack_1248.py	26071	b5030fbb14b38d358294832ee044a752025efa267290d93080c406960010815f	0
src/krux/pages/tc_code_verification.py	2805	6fa947fe4b5ad2d579151983c886118dfcaa888c0481eea0884eacd818c4ffd5	0
src/krux/pages/tiny_seed.py	33522	7dbb6f0bb6f735b15da24842945c134243cf8bfc0a57bf83cfba706391bea3c2	0
src/krux/pages/tools.py	4596	746f4b1bdba84b00df3e1b556b9e23b2e4a90ab85724ee2a048e09512f24ecef	0
src/krux/pages/utils.py	7276	e3c3da9baf689eaf63905f7aa222454b75c605493deebdb32033053281046e5e	0
src/krux/pages/wallet_settings.py	17567	61fea7327078b81bab4eb9b90ace3cb6dbb9c51d3e047fb67de329f76f9f4f47	0
src/krux/power.py	4043	b485193950130c7ca527329f4ff2ee95c2d698e3a0c50f0aed52a471011f1d16	0
src/krux/printers/__init__.py	2364	7939c1d5fe5fc2556dec8692c4a7c0bdcd9170d241a4b6938dfba08b7e21aa03	0
src/krux/printers/cnc.py	16890	b947fbe16cd96f01e80bef00e113c6f840f533dcd65ff85bc3910fd7fb727b20	0

src/krux/printers/thermal.py	7255	636cdc0d7b33e575006e7c3e3a16e2eaec668e90fa6591ed4592f02be002e0c4	0
src/krux/psbt.py	34012	f1c6d32a97d52a9aa6f4b748b6a2e57e4a995f70d34152003ff9256c42d7cf51	0
src/krux/qr.py	15165	e448e374e13dc81126fea500760d3285a07682eca8895005fef8e2dc73ed9ddc	0
src/krux/rotary.py	4005	32810d66b4c66537bcb1854137f551174b2fb70cd0ac7067d308d77df40881e6	0
src/krux/sats_vb.py	7690	02444ea87eb2769a97a969f6eebe581a748296a6b3585f743a3e2b9d6866da30	0
src/krux/sd_card.py	3436	2a96900f4740cc7607e4a5a70830c731f7315a2780accc51daf0010bc6b2c57b	0
src/krux/settings.py	11296	e970473ef04dcffd0162cf65559df92004360a7ce12aec10a33ea05b58e9ab2f	0
src/krux/themes.py	4474	4e3be110e0b747a991d84cc8ed71c93684ef009a1676209cb714e74dbe8ede40	0
src/krux/touch.py	10768	ff7e560c1bbf20ad03b14a4575ac026a1b0f42dd48990f3d25a6c48ff3585274	0
src/krux/touchscreens/__init__.py	1486	e571db078cbae903fa36f7b49b5a4ed8afbeb143f12bedc94ca879e9c6efc8f3	0
src/krux/touchscreens/cst816.py	5022	2262ca063871a3d815b5c13502395b911e36df0314a8dc9fc71940f165ef5f1f	0
src/krux/touchscreens/ft6x36.py	4079	9449387057ba70354d17eb5f17b9b1c9f8e5035da02c0b806c6c16c1a50ab0b2	0
src/krux/touchscreens/gt911.py	6832	5663e3195bb7d5ac486bb93010c8f4663a84c332f3f0e5f7cad272fb4c5f8c79	0
src/krux/translations/__init__.py	7463	43909e8d9788dd7a234843b218b16850422235c9a3d4827906ab039a22da4c92	0
src/krux/translations/de.py	12666	d9b99c4d81346055f922e8279e42537ad62a8be47e546967fb4a071398f9bf2d	0
src/krux/translations/es.py	12799	5883f2ea41fd5df350c6b90fc456181626f06f1f18eaf72f539e7131da1f16f3	0
src/krux/translations/fr.py	13585	9786bcee093df7393a413ed8a98e59a42645486633be858a8ce0c8e6d93900a4	0
src/krux/translations/ja.py	15357	6c359877fd74d165a5087f7eadb85c667634cc5a31cc1ec0812cae45cc5eaad2	0
src/krux/translations/ko.py	13997	ddb6ecc9dff600402f1f4284add7dd344a0c5a298df85663efcd3218dee4d3f3	0
src/krux/translations/nl.py	12421	536187ea00a2dbd2316bfa52bc3d6f1fe3711c299498e4e77afc05dae74043ee	0
src/krux/translations/pt.py	12661	9f5815a259ad88a5ab3554175f078d2b25db30f32bf62241bb023427a683dda1	0
src/krux/translations/ru.py	19120	583d5dc3331c9d37c03949150c9c568556f172361990edc6d0fd182997801201	0
src/krux/translations/tr.py	12665	42198615f8d2d7b700253f599b473911247eb14e8f1f2c6805603618d4adc6c4	0
src/krux/translations/vi.py	14132	e48e7f574e44366e2693c71397b377054d84a28ef60e4ea47616aab55bfe6b95	0
src/krux/translations/zh.py	10840	cbe33b1cdc21aeb196b0da9bc2db5d318156e81607e6af3f5fb48453fb628d0a	0
src/krux/wallet.py	21233	810d6df697e709814bb9d650ecf51e737ff61c0f20133d40dbc644cebd6403cd	0
src/krux/wdt.py	1277	63372e251d26ccea92c2c8d235f06bacd5f52c611267a71a5ec457c6cb8055c0	0
firmware/Kboot/build/ktool.py	135603	800794f7d621f44e393400d31b291c2f087d0cabf986dd35ae727eb33729dca5	0
firmware/Kboot/src/bootloader_hi/fpioa.c	108140	fe9bdd25e4404c6d37767038773edff55cdabf46c17fa35935685afb00c76023	0

firmware/Kboot/src/bootloader_hi/gpiohs.c	2838	e7b45f7d005b0220bcd48604745b9c0b7a2d675071721abb59a1a9713e167789	0
firmware/Kboot/src/bootloader_hi/include/encoding.h	46554	3ced07bdbd1389c035aed5422b64db521e17fee41e9a0796dd428a48ced0c88d	0
firmware/Kboot/src/bootloader_hi/include/fpioa.h	51938	8ed1e43f8c801d3f4e74d6b80b9ceece944b2f9520831326f0ff08b3175730a4	0
firmware/Kboot/src/bootloader_hi/include/gpiohs.h	6815	43f2758481477f89bc6c080f812f821435af0bf780304e275a697b7ae16984fb	0
firmware/Kboot/src/bootloader_hi/include/platform.h	2910	fbfc117bb58ad42e75560896a8d44bbf1b946a931e2ccc4ea82b19561081206e	0
firmware/Kboot/src/bootloader_hi/include/sha256.h	3597	ffa02fcca2254789838ad39c713a113e9643e3a6be80b7a0f3a159c64ac92234	0
firmware/Kboot/src/bootloader_hi/include/spi.h	4864	48dbd2d371dff0a68c2a79634a051f88af3b57004519ff7f6a6f211a6a24768d	0
firmware/Kboot/src/bootloader_hi/include/sysctl.h	24408	917554ed51202e48b82f6118ab03cb1abff881e163322e3e39fdf65dfd220383	0
firmware/Kboot/src/bootloader_hi/main.c	17753	e595d8d4a431bb6cf24e584ba4c75363916ed918efd78657a63eb6443ae3148e	0
firmware/Kboot/src/bootloader_hi/sha256.c	4381	faafd74ad01e75cf4fbace5b5db360fa6b2e13755a5c1eb729662756df495870	0
firmware/Kboot/src/bootloader_lo/include/encoding.h	46554	3ced07bdbd1389c035aed5422b64db521e17fee41e9a0796dd428a48ced0c88d	0
firmware/Kboot/src/bootloader_lo/include/platform.h	2910	fbfc117bb58ad42e75560896a8d44bbf1b946a931e2ccc4ea82b19561081206e	0
firmware/Kboot/src/bootloader_lo/include/spi.h	4864	48dbd2d371dff0a68c2a79634a051f88af3b57004519ff7f6a6f211a6a24768d	0
firmware/Kboot/src/bootloader_lo/include/sysctl.h	24408	917554ed51202e48b82f6118ab03cb1abff881e163322e3e39fdf65dfd220383	0
firmware/Kboot/src/bootloader_lo/main.c	7501	873afe5fe11e294dfa040fff18dbe8fbc74e59aeb8ecdd8834477b7b8d0740c3	0
vendor/embit/poetry.lock	141384	2cb04f5c9e548a06fd72f23071379937898e90680cf8323c9ad76b732067b4c9	0
vendor/embit/pyproject.toml	2272	13dbd4a607f5ab89d185fd59084e095ec07f61aff41ae796593140fe510f0e86	0

vendor/embit/secp256k1/config/libsecp256k1-config.h	1049	b89383e0d2b44a0ee82a1047507425812039ba509d5ef6c670b48bac2bfc5ddc	0
vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_parsing.c	3575	14225148538be5587db2304dcb6ba6340b2f195352ada8de78da53ba9ca7eec1	0
vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_parsing.h	4234	79c479a9e6f51a5ad1e83b9efcde9df4e3431c2a3e894d37de8e9c9bc4538fb2	0
vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_privatekey_parsing.c	5096	53acbaf3ab72f97855a87d936bfb32e288745de9d2311da8750e9ff842f905ee	0
vendor/embit/secp256k1/secp256k1-zkp/contrib/lax_der_privatekey_parsing.h	4048	3dcaf6d98270a20ef1455f67479191ff31747af1e66cec979b01a2be41a968f2	0
vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1.h	36956	524e48b39a62c9f54f203dca9b8a5a69946028fe7adeadeb60a81e590eb5c78a	0
vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_ecdh.h	2508	7950343b5eea7112a491850ad20fd0862ae965f84ae0e8e74ea404cf83e6d12a	0
vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_ecdsa_adaptor.h	7473	e2961621b8bd190b7c4426d10203876d33c2ab0c6a16adcb5f016da47fa36c70	0
vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_ecdsa_s2c.h	12308	981e83c3d35e25dcb292b0e9d7d07fa4093173a431d87c3efb5e587a9f75c449	0
vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_extrakeys.h	12104	d134316cf706bdbce101a8225d24c07c4d1774d4d64815fd6a8e743e0c2bba60	0
vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_generator.h	3430	05d5b4b7b8ff0954a3bfe50025093bbf9fd1aad6ecd0886b2042c9a671efd934	0
vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_musig.h	24450	b492db382be02ae4423f796c6f3a051d355009ea6dcc1948d50359232a30bdd5	0
vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_preallocated.h	5689	a398cf3eeb4d8041825e8798770447efcd3118771f4f58c1477877e01555ed0d	0
vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_rangeproof.h	15139	2a1e6b3ae4c44f3f453b1d53c996e617a4af1f38acd3b5e7635d4eb369aa0041	0
vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_recovery.h	4742	92b8b66ad249ca92a4dd41fed8ee56cfaf34bf29f1ac12398324f87157baf1e5	0

vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_schnorrsig.h	4941	4c06e835211e9426c0a1af5a6e7ba8a9b7a0849eb6c60adb9f9309902266b9b4	0
vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_surjectionproof.h	12419	a7003623b033602fff8abd1dbd8084a6c0e3c0f30a3fa3e2604d36a8b9945d1b	0
vendor/embit/secp256k1/secp256k1-zkp/include/secp256k1_whitelist.h	6507	2cb891357e7113721542c6cf61f7c32796f70a68589ef1103b691223b3f5231d	0
vendor/embit/secp256k1/secp256k1-zkp/src/assumptions.h	4366	1a7f40ea5b62146fa1809da4799e3315743195950b494b12ce2d0c81e8c76024	0
vendor/embit/secp256k1/secp256k1-zkp/src/basic-config.h	590	5af274e93a0de7a51f1111f356d249262b1934976d0e537c072ddbbbec2ae7b6	0
vendor/embit/secp256k1/secp256k1-zkp/src/eccommit.h	2162	b9336fd346fcde5886597c7c8b34e0fa9769519774c093695341e1d60a7d4431	0
vendor/embit/secp256k1/secp256k1-zkp/src/eccommit_impl.h	3117	7cb79dcac87f70f59664dd0599e6f79f1f5f5b779e1ca3e4c599d1eb16309dad	0
vendor/embit/secp256k1/secp256k1-zkp/src/ecdsa.h	1201	812552ffa5a48b516a7dea910a15278739cf1376dbcc650ccb3d4dc29269d824	0
vendor/embit/secp256k1/secp256k1-zkp/src/ecdsa_impl.h	10686	475d3dd3d6d4338201468465b88c08f8cf8cc7ff6c610d7cd3a03721bc01c9f7	0
vendor/embit/secp256k1/secp256k1-zkp/src/eckey.h	1237	4638e4e797460ede9b9dd78b1306cfa17a8a9ba5828433325773722acc80ccaa	0
vendor/embit/secp256k1/secp256k1-zkp/src/eckey_impl.h	3464	2a6e70bc9b74cd0c10236a17c1dc35a72f0dddbc33906ad945aae4a9fe364627	0
vendor/embit/secp256k1/secp256k1-zkp/src/ecmult.h	2487	7fe6bd0b0752025015e54982125d655893c465b4e93a1f8eeaee169f3dd6756d	0
vendor/embit/secp256k1/secp256k1-zkp/src/ecmult_const.h	853	2d4839089f63eb4993987262b9bf9455f6c2b660789ce1fadf37867bd283bfcb	0
vendor/embit/secp256k1/secp256k1-zkp/src/ecmult_const_impl.h	10274	e9b3796ea1fdd731a9b1d80f5d9ae9d9e8784560f705ea13ad7ce47eb737f77a	0
vendor/embit/secp256k1/secp256k1-zkp/src/ecmult_gen.h	2708	a7c4ae33d19fe563377c5eace565555fd8bb9b183c52e39072f84f6f5cabe831	0
vendor/embit/secp256k1/secp256k1-zkp/src/ecmult_gen_impl.h	8857	e7e3d7fb0881bd5f511eea4ba90ccc8ad54c357505dedadd2100a1ab428841dc	0

vendor/embit/secp256k1/secp256k1-zkp/src/ecmult_impl.h	44043	f4dc1435edb6dad53ab4f0f5706855db45e45a6baafd48cffef3e17952f219e7	0
vendor/embit/secp256k1/secp256k1-zkp/src/field.h	6213	d499ca2108b4271a81896140d1490df9b825291311eeb2e99d103ced939189c6	0
vendor/embit/secp256k1/secp256k1-zkp/src/field_10x26.h	1987	ef897ddfc9b1c3e15eb08700808222b4cfdb5c6596c7e954df1fb24fb6ebe33f	0
vendor/embit/secp256k1/secp256k1-zkp/src/field_10x26_impl.h	50619	3cf50cb54d0f4b4db0cbeb4266a3d9333f5219203aa18449e9f9276088688930	0
vendor/embit/secp256k1/secp256k1-zkp/src/field_5x52.h	2030	4c016d20cc779b03d46413ecfdadb4cab06bba412aa365648e48c8c4b57737c3	0
vendor/embit/secp256k1/secp256k1-zkp/src/field_5x52_asm_impl.h	13167	c1e68e895926261022aeb7bc08eb4794231149b50a0cc79b37033cbc77cd1174	0
vendor/embit/secp256k1/secp256k1-zkp/src/field_5x52_impl.h	19115	7cb7c821d7f63658c2310e0366e38b9f078a88220704b75a713d6103fbdedc27	0
vendor/embit/secp256k1/secp256k1-zkp/src/field_5x52_int128_impl.h	9349	0d7f1365e76aec0af251f8e6a99cf59cc81b02e0b565a35e73ce8a4097e668ca	0
vendor/embit/secp256k1/secp256k1-zkp/src/field_impl.h	4176	98391a3485187a4ca2af56661412f1329bc764cd131985a177cb8b6801009e4e	0
vendor/embit/secp256k1/secp256k1-zkp/src/gen_context.c	3515	475071487ef772208fd3bee8cdce40ca536fb34c90b093f091285d87d27d88bb	0
vendor/embit/secp256k1/secp256k1-zkp/src/group.h	8502	3c12f857092bf57c103479a5c49b4b59eedbc3bc61682bbd526d9722dd82dfff	0
vendor/embit/secp256k1/secp256k1-zkp/src/group_impl.h	27300	cc9fdd12aa8cffb17cf96d2ac0faa1213b1a746c69afa42239bb6b46bba6834a	0
vendor/embit/secp256k1/secp256k1-zkp/src/hash.h	1714	038dcdfcc50e4a99ecf9aabde34494f218a4e791405c233c59119bdd5b7ce756	0
vendor/embit/secp256k1/secp256k1-zkp/src/hash_impl.h	13381	dcaccfbbf8001e28f728237e09cf2064518e66d3de848ee7697d834d5ef98251	0
vendor/embit/secp256k1/secp256k1-zkp/src/modinv32.h	1590	547dc92e3d8d5d66748d2de4b1dc1559d33c3ce8de62ed4be5b6dbaad29c43d4	0
vendor/embit/secp256k1/secp256k1-zkp/src/modinv32_impl.h	26705	0a74faf781c312e93b58d7f00f2b2a8a76882794b029609f52b1d1eb8d2bb095	0

vendor/embit/secp256k1/secp256k1-zkp/src/modinv64.h	1694	d5fb8ac55ef964df7cd9d03c48b07e39bf25d6c5be01f2aefe7e5bbdb893607c	0
vendor/embit/secp256k1/secp256k1-zkp/src/modinv64_impl.h	27997	4cd24aa3b4a59b96fe5a2be212b4ec9ac511b6bf49af7532aec529f79593ceb2	0
vendor/embit/secp256k1/secp256k1-zkp/src/modules/ecdh/main_impl.h	2361	b88931b1b2f71adb0ef917064910601240f5a380b45defc0759b75e8e878279d	0
vendor/embit/secp256k1/secp256k1-zkp/src/modules/ecdsa_adaptor/dleq_impl.h	5642	7d5652b63ec268bf08486b01ef0ce3de6ada5629937da1b4bfa7213b76064671	0
vendor/embit/secp256k1/secp256k1-zkp/src/modules/ecdsa_adaptor/main_impl.h	14327	177349a8e2d98112868ebb4eb0d9a5c883a4ce09d8ef4866e78f4059bddaeec6	0
vendor/embit/secp256k1/secp256k1-zkp/src/modules/ecdsa_s2c/main_impl.h	8616	723bdad7d125439572a67341d0bde2275ecd94f23f7bbf8c543274956ae5f22d	0
vendor/embit/secp256k1/secp256k1-zkp/src/modules/extrakeys/main_impl.h	10189	0c88a343e58e38a909be3a62f0a7023b4231e24071d8df50ee0cd2dde2d930cd	0
vendor/embit/secp256k1/secp256k1-zkp/src/modules/generator/main_impl.h	8233	0ad192c8973c4da4c9980a573c415c78ea54404c3fa8200839cd20e43f03f50c	0
vendor/embit/secp256k1/secp256k1-zkp/src/modules/musig/example.c	6918	7568f0ec3778da013a98fd5e3133c2a640e3eb3f3cff59290cee43de7cdf856a	0
vendor/embit/secp256k1/secp256k1-zkp/src/modules/musig/main_impl.h	26302	f5b1e1cb467ee05a70006c55efe00cbc54962adf5a5446f74438f0b282aa1d75	0
vendor/embit/secp256k1/secp256k1-zkp/src/modules/rangeproof/borromean.h	1143	668ad333886c5a7db3f518cdc9acf88fc78eaad04e6d280fcb0ee58a571b94ee	0
vendor/embit/secp256k1/secp256k1-zkp/src/modules/rangeproof/borromean_impl.h	7996	970fe56254e3e90c7bc14aff93c181ea96c35b22465a1ae15b8bc389d5174cf5	0
vendor/embit/secp256k1/secp256k1-zkp/src/modules/rangeproof/main_impl.h	12799	6c46c3678bdc43b3f85275e27b7e05dad3c757069443068e8d56080d3f9fd8c4	0
vendor/embit/secp256k1/secp256k1-zkp/src/modules/rangeproof/pedersen.h	890	ba1740c13e0e28450fdc7c2222b4dcb535979a632ca418ab3b7499c459f827e0	0

vendor/embit/secp256k1/secp256k1-zkp/src/modules/rangeproof/pedersen_impl.h	1677	241097c4cd4bf909821b34e6d0da590f3409f0703f061734ea19fdfdbc67c6ae	0
vendor/embit/secp256k1/secp256k1-zkp/src/modules/rangeproof/rangeproof.h	981	f136150e8e7347ff0bfac863581d9d0506bf1d4f528bf29750a7db4d2d4a8b54	0
vendor/embit/secp256k1/secp256k1-zkp/src/modules/rangeproof/rangeproof_impl.h	25672	02ac85491617607e7f38e073d1f4769ddbcfd74603b460fc1598d61aff2feb51	0
vendor/embit/secp256k1/secp256k1-zkp/src/modules/recovery/main_impl.h	6227	d130fb409ac92c96a2ec3d368998cc6de277bbdb7e11e5ab1234d8aaca17834e	0
vendor/embit/secp256k1/secp256k1-zkp/src/modules/schnorrsig/main_impl.h	8187	eb51e55a0617e6145afed8ccdd2be86d3fc640a0a996f421a1caa5eb3f34e935	0
vendor/embit/secp256k1/secp256k1-zkp/src/modules/surjection/main_impl.h	15987	44294734be933d3236040e7f690de0a28783ba7747a729ba4b9d3c88c141aeca	0
vendor/embit/secp256k1/secp256k1-zkp/src/modules/surjection/surjection.h	1058	33eb9735803f1dbcb20ad42370f011f3d876e49e5d0d5998013fd644ec466fc0	0
vendor/embit/secp256k1/secp256k1-zkp/src/modules/surjection/surjection_impl.h	3448	fa0a740afca1ef6a9bfd5776166692a885b67272bcdc46bb3b46f9555de7d574	0
vendor/embit/secp256k1/secp256k1-zkp/src/modules/whitelist/main_impl.h	6489	7c4ddad42f296606dd12d2ae683345f300e9a3c846dfe60265ab9c0661332e69	0
vendor/embit/secp256k1/secp256k1-zkp/src/modules/whitelist/whitelist_impl.h	4527	3a673bb954f54cf5c0a511efc6628e4e5512047b7d2580efd58e31d81df7ed62	0
vendor/embit/secp256k1/secp256k1-zkp/src/scalar.h	5430	41f80a5baf85ef8ad70fced31d3001b6c82ccd6400835c352de8277db2eb21e1	0
vendor/embit/secp256k1/secp256k1-zkp/src/scalar_4x64.h	801	42aed583b28bda7982c54a6f63e50c0ae5f9c27d5f66d92ec6bf6c90e765735a	0
vendor/embit/secp256k1/secp256k1-zkp/src/scalar_4x64_impl.h	31469	0ab498d00f7f0cb2b7d5afb350e1d571425c5be56e6c124144084bfe4d41a5ac	0
vendor/embit/secp256k1/secp256k1-zkp/src/scalar_8x32.h	725	0614cea113aea88a47ef3b2ba8167722e850d7b2a696a4f16b03ec9a320ef1d6	0
vendor/embit/secp256k1/secp256k1-zkp/src/scalar_8x32_impl.h	30373	9cc6e80eaeb189290f07ea2aa115426b5337e8cfa96bb9d5157acc842691a328	0

vendor/embit/secp256k1/secp256k1-zkp/src/scalar_impl.h	11302	f930431bd0739551cb40949e4816cde21880cf33c4d7c0d2e68466b00ddb9010	0
vendor/embit/secp256k1/secp256k1-zkp/src/scalar_low.h	658	f0427de1e87d0f4e06f5dc1d23d036056afab26e809cfdec5b16e45d946be477	0
vendor/embit/secp256k1/secp256k1-zkp/src/scalar_low_impl.h	5047	344191609736ed83d118b6343c881c94e8883fa340f6b9d9270cce4ab34e84eb	0
vendor/embit/secp256k1/secp256k1-zkp/src/scratch.h	2152	0301e4ae76f1090462b1e798bc0b4f25359c0ee38dd3e10f06f2fd491e2ffb9e	0
vendor/embit/secp256k1/secp256k1-zkp/src/scratch_impl.h	3677	b7ef71c35713908ea32aab65ed5a42a2be13bc7b5a0fc2115d891dce7fef9125	0
vendor/embit/secp256k1/secp256k1-zkp/src/secp256k1.c	31640	c206ee81472c00de7c947edbb1178e22bb3bdcbff71439a0440f3d8072b6e9f1	0
vendor/embit/secp256k1/secp256k1-zkp/src/selftest.h	1264	4deb00dd293926cc8a52aab5f59351c6fd25d72dd26c448dcc3ec6d16d370a55	0
vendor/embit/secp256k1/secp256k1-zkp/src/testrand.h	2030	e1aab215d46d5168a2e411663cfc1f00bcdd15419d115baf8457608886957440	0
vendor/embit/secp256k1/secp256k1-zkp/src/testrand_impl.h	6500	6fc1e8956367d5d7fbf86cf5fe4d30ba28cee9ed0fb93939f9c7c30f9b252907	0
vendor/embit/secp256k1/secp256k1-zkp/src/util.h	13326	c987ae9066b447e16c102767b8cd58b2555b5db4a0cd966e9bf4a58ac4c983d1	0
vendor/embit/setup.py	63	cac35ae0415825d9140c994aac5cf0e4178f180bf32cbc86d8be4913a00937b4	0
vendor/embit/src/embit/__init__.py	0	e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855	0
vendor/embit/src/embit/base.py	3472	7c0b75fa140abce6663fc879f811c69bc231099b962f968ed362ac1bfbe10461	0
vendor/embit/src/embit/base58.py	1978	9166fd6cec484bf1c14a4ca1dabc05a9cff1613a35598737945da0b5dec81d5f	0
vendor/embit/src/embit/bech32.py	5165	9ad80ea66ef64f7e2c8906175fdd5bc3ba19952fea6a356edd15cf693736ba3f	0
vendor/embit/src/embit/bip32.py	10157	44cbc51b66db1ee9651545d899da381b11da778c5c8f63f03de03bc1075726bf	0
vendor/embit/src/embit/bip39.py	4140	630534f8fb4a6559eb8492c124f581a8530dae40137cd53ec74047e192a83429	0
vendor/embit/src/embit/bip85.py	1910	9ebb91c3a22f5f10de08456d83927fea9be2191bee833782c63d90c91d91310b	0

vendor/embit/src/embit/compact.py	1135	558255ead2d0c05c8c2241de6906717cee946a31ee35a1c8d99d4a3e1bb50cd8	0
vendor/embit/src/embit/descriptor/__init__.py	87	4aa84fc0a3ed296e6dc2da33d5363444b1a5999883a0a8724caaf4acc16abe29	0
vendor/embit/src/embit/descriptor/arguments.py	16154	c6bfb0a18831955fc2718e0657ac9cef966572abf37fc588e4d977268b73ca99	0
vendor/embit/src/embit/descriptor/base.py	606	28f0a4f67943598e5cec7e108bb173c09dbc549828cfc4f5b448aaf5fbc9b8a1	0
vendor/embit/src/embit/descriptor/checksum.py	1424	ce2f12172291964693395525bcff945ca2c13703e19343dc5a102765c75f8ef3	0
vendor/embit/src/embit/descriptor/descriptor.py	12227	e5083d56f2c6aa5158b6491df835bcc706b08c911ce5e978007b6eb420246f53	0
vendor/embit/src/embit/descriptor/errors.py	218	b168765207c4248fbe3abfca5c1117871f988452255b2d7d01db3fd73d84e589	0
vendor/embit/src/embit/descriptor/miniscript.py	27644	7ed80636da6dc861b1ec703fbbaca4bf58acaa2f5a71b012b2881018e56cfe77	0
vendor/embit/src/embit/descriptor/taptree.py	4775	1a5dc80eedaaf07d3de85417e320de529cf9283aadc752f8b7d5874b38dea826	0
vendor/embit/src/embit/ec.py	8332	1c2b11a8e397a903473962ff0baa2b3de02e81c211b6df300fe828f1249a533b	0
vendor/embit/src/embit/finalizer.py	3339	0a1b2be13e3c9e41ca7aa78ab7c5b67dc81f6571fe8e849252d25be261853520	0
vendor/embit/src/embit/hashes.py	1222	a0fbfa6dfd9c2db718a706e20b09bbf29d0ff120d8381f2db0477f8f169b9505	0
vendor/embit/src/embit/liquid/__init__.py	0	e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855	0
vendor/embit/src/embit/liquid/addresses.py	3182	7560dad8a486ba99ef94507ce0b2598ffdcce7e3ac94f76dda7b227f18d946fa	0
vendor/embit/src/embit/liquid/blech32.py	4555	f86db4a16f3aae7b21bccd084f228dc3b8964aecbcf314195a4aca61cc0591d6	0
vendor/embit/src/embit/liquid/blip32.py	481	62d971910564b0174001a5bd0669733d5c8ef8459ce1b8cbe959ff0bce454437	0
vendor/embit/src/embit/liquid/descriptor.py	8775	faeaf423b09b7889ce4e47e93cd3563358f7cfa39230af015ea4ef7a1b47bafd	0
vendor/embit/src/embit/liquid/finalizer.py	2009	8977b650b2cfcba3222ddc8b01a1279faf5b398064168c733f5d03ac9d9c7d9f	0
vendor/embit/src/embit/liquid/networks.py	2145	ffb9ded28c4dd810cffa57fde65f742d0c6deb5dba9ff757d92e0156aa5e814a	0
vendor/embit/src/embit/liquid/pset.py	25766	b9bd8b713c6395282122052daeb4e95540d556abcaf40e055cb9c597d618738b	0

vendor/embit/src/embit/liquid/psetview.py	9114	ca4aaf43aa5e241e807ec47ec2b3d166c4aff1479a533de03319073e24c46b15	0
vendor/embit/src/embit/liquid/slip77.py	453	0e797e76b309e09b7323ee71e86e490bbbbad3695318c732308cd27561b44aeb	0
vendor/embit/src/embit/liquid/transaction.py	18550	601e672e0ae53c08780b307a0b231215d31c8325b0fe18b97bb45a3e2892cb16	0
vendor/embit/src/embit/misc.py	1820	fc544d7dcb476386b4a2c1b47ea6c6da553e3471d916e8b2c24ea6328769ef9d	0
vendor/embit/src/embit/networks.py	2256	d1ba1531c9bb5fb67e838b092a2f30153a8a3b135c61c4819ab20ea1339bd94b	0
vendor/embit/src/embit/psbt.py	42662	fdd1fee6816ec082b0cc61ef8d772406c4f5f41f06c8f58ac5adfcc83e53fa1a	0
vendor/embit/src/embit/psbtview.py	35129	e6eb9fc726c801fcb157a15d8328dc6dabe816cca28b43bc8a34be10c3ced0b8	0
vendor/embit/src/embit/script.py	6354	f35827d59a038ba9b0d11ab80697ae3f5913f6bf16d7c078b627180f5ef651e3	0
vendor/embit/src/embit/slip39.py	13751	363e0dbd9282b7d6bd0e6aecaf04b58a3d86dd69df05eaaa1cf7df82870add79	0
vendor/embit/src/embit/transaction.py	14415	b43f0974608332268ef5fc61467fd5918f2b5015e0bc2baa7e842dbd08d05653	0
vendor/embit/src/embit/util/__init__.py	96	23cccc8f03e645102b5d2722758ac955edda2a2b14b7d24eec17a28b71595c79	0
vendor/embit/src/embit/util/ctypes_secp256k1.py	39495	76695a82303a769942a1d9edab23aaa9380af577ebae6ddf2eae04acc6b5d43b	0
vendor/embit/src/embit/util/key.py	20166	0850b1637dafc13c7c2e0d1e4afbbcbc655890dc142da543c340ac8e494a5dd9	0
vendor/embit/src/embit/util/py_ripemd160.py	5397	44d730f6ced0ee5d4ffaa9662420779f8797bf5ba3a1927dd11b6af2639a08d3	0
vendor/embit/src/embit/util/py_secp256k1.py	13341	cd71a7d933635ba82eea52e9af59854ab03392e9eb866ff1072f581c3de716ba	0
vendor/embit/src/embit/util/secp256k1.py	3751	3f0dfccabac80a6c83c0d2318582d296d0fa9cf88ab347418ba7e27af5e333c6	0
vendor/embit/src/embit/wordlists/__init__.py	0	e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855	0
vendor/embit/src/embit/wordlists/base.py	897	c4e144c64cbc7e46e8c9e66ad625cc4e2a04f3a13d71ae2c197413973e79ace1	0
vendor/embit/src/embit/wordlists/bip39.py	27467	c7398e797b743901822741e5a2c6010cb1ac09f7ea1bc562d6da0d8bf4d164e2	0
vendor/embit/src/embit/wordlists/slip39.py	14418	9141a27eb0a1c62be940660318966ae9965358a5a517317ac67b75ef47e1f502	0

vendor/embit/src/embit/wordlists/ubip39.py	124	5bb74513560d9643773b3e4c6016094414c2c6bbc7bfe97944e66dfbf27300be	0
vendor/embit/src/embit/wordlists/uslip39.py	129	368d0994125fd8124c26b6009042dbdb153b9da011cbc6bd509df7b6cb9e4241	0
vendor/embit/tools/generate_release_sbom.py	5351	abe044b870fa5b5b86be6eeff07e9ff1abffc969644cf9093ddc4c9f2b7f0ee9	0
vendor/embit/tools/package_inspect.py	856	30e1e125db2ab21d39b6b93bc2f216c28eb948472aaec61f69443c33748d60fd	0
vendor/embit/tools/verify_github_actions_pins.py	6600	79f0b6469a0c964f7699b477939a8abfc6831fa545f9d149b0b17e769c234eed	0
vendor/embit/tools/verify_package_artifacts.py	12821	f57fa3c5279910e6d63944a55b6b40f80d12869ed85d0e4931ccb87ded2d17d6	0